



SPECSFY

Guia completo do usuário

Specify. Prove. Ship.

Da primeira ideia à entrega comprovada, em uma
jornada prática e rastreável.

Edição v1.2.0

Gerado em 28 de julho de 2026

Fonte editorial: docs/user/

Sumário

Guia completo do usuário	9
Leia online ou como ebook	9
Percurso pedagógico	9
Conversa contínua entre etapas	11
A ideia central em um exemplo	11
Como a metodologia funciona	12
Uma única especificação	12
Da ideia até o código	12
Os três atos	13
Como BDD e TDD trabalham juntos	15
O agente conduz as transições	15
Mudanças durante o trabalho	16
Contexto que permanece entre entregas	16
O que você encontra ao final	16
Limites do método	16
Instalação do Specsfy	18
1. Instale o CLI	18
2. Prepare seu projeto	18
3. Confirme que está pronto	19
Se algo não funcionar	19
Próximo passo	19
Uso básico do Specsfy	20
Papel	20
Como usar	20
Veja o Specsfy trabalhando	20
Resultado esperado	24
Limites	24
Próximos passos	24
Atualize quando	24
Não use para	24
Fonte da verdade e precedência	24
Caixa de entrada de ideias	25
Capturar	25

O que o arquivo organiza	25
Ideia, backlog e spec	26
Templates instalados	26
Backlog de requisitos e promoção de ideias	27
Papel	27
Como usar	27
Atualize quando	27
Não use para	27
Fonte da verdade e precedência	27
Estrutura	28
Organização do backlog	28
Anatomia de um item	28
Padrões recorrentes	30
Priorização	31
Fluxo	31
Estados do backlog	32
Quando está refinado	32
Limites	32
Implementação executável	33
Justificativa de tamanho	33
Skills base	34
Como escolher	34
Capturar uma ideia com <code>specsfy-base-idea</code>	36
Quando usar	36
Como pedir	36
Exemplo passo a passo	36
O que esperar	36
Erros comuns	37
Próximo passo	37
Guardar uma ideia com <code>specsfy-base-backlog</code>	38
Quando usar	38
Como pedir	38
Exemplo passo a passo	38
O que esperar	39
Erros comuns	39

Próximo passo	39
Aprofundar uma ideia com <code>specsfy-base-interview</code>	40
Quando usar	40
Como pedir	40
Exemplo passo a passo	40
O que esperar	41
Erros comuns	41
Próximo passo	41
Criar a especificação com <code>specsfy-base-specify</code>	42
Quando usar	42
Como pedir	42
Exemplo passo a passo	42
O que esperar	43
Erros comuns	43
Próximo passo	43
Validar a definição com <code>specsfy-base-validate</code>	44
Quando usar	44
Como pedir	44
Exemplo passo a passo	44
O que esperar	44
Erros comuns	45
Próximo passo	45
Planejar tarefas com <code>specsfy-base-tasks</code>	46
Quando usar	46
Como pedir	46
Exemplo passo a passo	46
O que esperar	46
Erros comuns	47
Próximo passo	47
Preparar testes com <code>specsfy-base-tdd-bdd</code>	48
Quando usar	48
Como pedir	48
Exemplo passo a passo	48
O que esperar	48
Erros comuns	49

Próximo passo	49
Entregar código com <code>specsfy-base-implement</code>	50
Quando usar	50
Como pedir	50
Exemplo passo a passo	50
O que esperar	50
Erros comuns	51
Próximo passo	51
Incorporar mudanças com <code>specsfy-base-update-spec</code>	52
Quando usar	52
Como pedir	52
Exemplo passo a passo	52
O que esperar	52
Erros comuns	53
Próximo passo	53
Consultar o estado com <code>specsfy-base-progress</code>	54
Quando usar	54
Como pedir	54
Exemplo passo a passo	54
O que esperar	54
Erros comuns	55
Próximo passo	55
CLI e TUI do Specsify	56
Papel	56
Como usar	56
Atualização automática	57
Dashboard e progresso	57
Testes do projeto	61
Justificativa de tamanho	63
Atualize quando	63
Não use para	63
Fonte da verdade e precedência	63
Segurança e reversibilidade	63
Contexto persistente do projeto	65
Papel	65

Como usar	65
Monitoramento durante mudanças	66
Atualize quando	66
Não use para	66
Fonte da verdade e precedência	67
Preservação e atualização	67
Documentação técnica do sistema	68
Papel	68
Como usar	68
Atualize quando	69
Não use para	69
Fonte da verdade e precedência	69
Atualizar uma especificação	70
Papel	70
Como usar	70
O que acontece	70
Resultado esperado	71
Limites	71
Atualize quando	71
Não use para	71
Fonte da verdade e precedência	72
Skills especialistas	73
Papel	73
Como usar	73
Atualize quando	73
Não use para	73
Fonte da verdade e precedência	74
Catálogo por domínio	74
Relação com as bases	74
Uso avançado do Specsify	75
Papel	75
Como usar	75
Detecte e instale contexto técnico	75
Automatize a leitura de progresso	76
Ajuste a atualização visual	76

Atualize sem perder customizações	76
Reabra a entrada correta	76
Resultado esperado	77
Limites	77
Atualize quando	77
Não use para	77
Fonte da verdade e precedência	77
Usar Specsify com Laravel	78
Papel	78
Como usar	78
Instalação	78
Passo a passo de uso	78
O que o especialista acrescenta	79
Resultado esperado	79
Limites	79
Atualize quando	79
Não use para	79
Fonte da verdade e precedência	80
Usar Specsify com Astro	81
Papel	81
Como usar	81
Instalação	81
Passo a passo de uso	81
O que o especialista acrescenta	81
Resultado esperado	82
Limites	82
Atualize quando	82
Não use para	82
Fonte da verdade e precedência	82
Usar Specsify com Next.js	83
Papel	83
Como usar	83
Instalação	83
Passo a passo de uso	83
O que o especialista acrescenta	84

Resultado esperado	84
Limites	84
Atualize quando	84
Não use para	84
Fonte da verdade e precedência	84
Créditos do Specsfy	85
Papel	85
Como usar	85
Projeto e manutenção	85
Comunidade	85
Identidade	85
Componentes relacionados	85
Como contribuir	85
Atualize quando	86
Não use para	86
Fonte da verdade e precedência	86

Guia completo do usuário



O Specsify ajuda você a transformar uma ideia em software testado sem espalhar requisitos, planos e tarefas por vários arquivos. Você conversa normalmente com o agente; as skills organizam o trabalho e mantêm uma única especificação como referência.

Este guia ensina primeiro a lógica da metodologia, depois prepara o ambiente, acompanha a primeira entrega e, por fim, apresenta operação e recursos avançados. Você não precisa conhecer a implementação do framework.

Leia online ou como ebook

Este mesmo percurso está disponível na edição portátil **v1.2.0**:

- PDF, para leitura, compartilhamento e impressão;
- EPUB, para leitores digitais com fonte e tamanho ajustáveis.

Os dois formatos são reconstruídos a partir destas páginas. A versão vigente e os hashes verificáveis ficam na [pasta do ebook](#).

Percurso pedagógico

Siga as etapas abaixo na primeira leitura. Depois, use esta página como mapa para voltar diretamente ao assunto de que precisar.

1. Entenda a metodologia

Comece pela [Metodologia](#). Ali você aprende a ideia central: cada entrega mantém problema, requisitos, exemplos de comportamento, plano técnico, tarefas, testes e evidências em uma única `spec.md`.

O trabalho acontece em três atos:

1. **Ato I — Definir:** entender e validar o que deve ser entregue.
2. **Ato II — Projetar e provar:** preparar tarefas e obter o RED, a falha esperada antes da implementação.
3. **Ato III — Entregar e validar:** implementar, obter testes verdes e registrar evidências.

2. Instale o Specsify

Com o método entendido, siga a [Instalação](#) para instalar o CLI e preparar um projeto consumidor. A instalação cria a estrutura necessária para trabalhar com specs, contexto e skills sem transformar este monorepo em um projeto consumidor.

3. Faça a primeira entrega

Use [Primeiro projeto](#) como tutorial guiado. Ele leva de um pedido inicial até uma entrega validada sem exigir que você decore cada skill.

Se ainda não quiser iniciar a especificação:

- preserve um texto sem perguntas na [Caixa de entrada de ideias](#);
- refine e priorize uma proposta no [Backlog](#).

4. Aprofunde o fluxo base

O índice de [Skills base](#) apresenta o fluxo completo. Leia cada etapa nesta ordem:

1. [Capturar uma ideia](#);
2. [Refinar no backlog](#);
3. [Conduzir a entrevista](#);
4. [Criar a especificação](#);
5. [Validar a definição](#);
6. [Preparar as tarefas](#);
7. [Preparar TDD e BDD](#);
8. [Implementar](#);
9. [Atualizar a especificação](#);
10. [Consultar o progresso](#).

Essas páginas explicam quando usar cada skill, como pedir em linguagem natural, o resultado esperado, os erros comuns e o próximo passo.

5. Opere o projeto no dia a dia

Depois da primeira entrega, aprofunde somente o que fizer parte da sua rotina:

- [CLI e TUI](#): comandos, interface visual e acompanhamento;
- [Contexto do projeto](#): stack, regras, banco e convenções;
- [Documentação do sistema](#): documentação técnica derivada da aplicação;
- [Mudanças posteriores](#): como incorporar um novo pedido à mesma especificação.

6. Avance quando precisar

Os próximos guias são opcionais e fazem mais sentido depois que o fluxo base já estiver familiar:

- [Especialistas](#), para conhecimento técnico adicional;
- [Uso avançado](#), para automação e integrações;
- aplicação em projetos [Laravel](#), [Astro](#) ou [Next.js](#);
- Mapa técnico, para conhecer os módulos do monorepo;
- [Créditos](#), para autoria e identidade do projeto.

Se você pretende contribuir ou modificar o próprio framework, continue no [guia técnico](#). Ele é um percurso separado do uso em projetos consumidores.

Conversa contínua entre etapas

Quando uma etapa depende de outra skill, o agente anuncia a transição, resolve a pendência e retoma o trabalho na mesma conversa. Você não precisa repetir o pedido nem conduzir cada passagem manualmente.

A ideia central em um exemplo

Imagine uma página de boas-vindas. Você pode preservar a ideia, refiná-la no backlog e promovê-la até chegar a:

```
specs/specs/0001-pagina-boas-vindas/spec.md
```

Em seguida, o agente valida a definição, organiza tarefas, prepara testes, implementa e registra evidências nesse mesmo arquivo. Se depois surgir um botão novo, a mudança retorna à mesma `spec.md` e reabre apenas os atos afetados. Não são criados `plan.md`, `tasks.md` ou documentos normativos paralelos.

Para começar esse percurso com orientação passo a passo, siga agora [a Metodologia](#).

Como a metodologia funciona

O Specsfy ajuda você a transformar um pedido em uma entrega comprovada. Você explica o que precisa; o agente organiza definição, plano, testes e implementação.

A metodologia existe para responder, durante todo o trabalho, a três perguntas:

1. **O que será entregue?**
2. **Como sabemos que o plano está pronto?**
3. **Qual evidência comprova que a entrega funciona?**

Você não precisa decorar comandos nem escolher cada skill. O agente identifica a etapa atual, anuncia as transições e mantém o trabalho na mesma conversa.

Uma única especificação

Cada entrega escolhida possui uma única fonte de referência:

```
specs/specs/<NNNN>-<slug>/spec.md
```

`NNNN` é o número da entrega, como `0001`. O `slug` é um nome curto que ajuda a reconhecer o assunto, como `recuperar-senha`.

Essa `spec.md` reúne:

- o problema e o resultado esperado;
- quem será afetado e quais regras precisam ser respeitadas;
- histórias, requisitos e limites;
- exemplos de comportamento escritos em BDD;
- decisões, riscos e plano técnico;
- tarefas, testes e evidências da execução;
- gates e estado atual da entrega.

Plano e tarefas ficam dentro da própria spec. O Specsfy não cria `plan.md` ou `tasks.md`. Assim, um arquivo não fica contando uma versão antiga da entrega.

Da ideia até o código

Nem todo texto precisa virar uma entrega imediatamente. O destino depende do quanto você já decidiu:

- Uma **ideia** é uma captura rápida. Ela preserva seu texto em `specs/ideias/` sem perguntas e sem autorizar implementação.

- O **backlog** guarda algo que você quer organizar e refinar, mas ainda não escolheu como entrega. Ele vive em `specs/backlog/`.
- A **spec** representa uma entrega escolhida. A partir dela, definição, plano, testes, código e evidências passam a seguir o mesmo contrato.

O percurso mais completo fica assim:

ideia → backlog → entrevista → spec → plano e RED → código → validação

Você também pode começar com “implemente recuperação de senha”. O agente verifica o que falta e conduz as etapas necessárias antes de mexer no código.

Os três atos

Os atos são grupos de trabalho. Cada um termina com um **gate**, que é um ponto de controle baseado em evidência. Um **gate** aprovado não significa apenas “parece bom”: significa que as condições daquela etapa foram verificadas.

Antes dos atos: escolha o destino da ideia

Objetivo: preservar o pedido sem forçar uma entrega antes da hora.

Sua participação: você informa a ideia e decide quando vale refiná-la ou promovê-la. Se pedir apenas para capturar, o agente não inicia uma entrevista.

Prova técnica: a captura recebe um arquivo próprio em `specs/ideias/`. Se for escolhida para refinamento, passa ao backlog. Somente uma promoção intencional cria a `spec.md` normativa.

Essa separação mantém a caixa de entrada leve e impede que toda observação se transforme em código ou em uma especificação extensa.

Ato I — Definir o que precisa mudar

Objetivo: entender o problema e transformar a intenção em comportamento que possa ser conferido.

Sua participação: você responde apenas às dúvidas que realmente mudam a entrega. O agente reaproveita tudo o que já foi dito, pergunta uma lacuna importante por vez e não inventa requisitos quando faltar uma decisão.

Prova técnica: a spec registra finalidade, pessoas afetadas, requisitos, limites e cenários BDD. A validação procura contradições, decisões em aberto e dúvidas prioritárias.

O ato termina com:

Definition Gate: Passed

Isso quer dizer que a definição está clara o suficiente para orientar um plano. Ainda não quer dizer que a solução foi implementada.

Ato II — Planejar e provar antes de implementar

Objetivo: decidir como a mudança será construída e demonstrar que os testes conseguem detectar a ausência do novo comportamento.

Sua participação: você decide quando houver uma escolha material de produto, risco ou estratégia. Detalhes técnicos comprováveis podem ser derivados do código, da stack e das regras do projeto.

Prova técnica: o agente registra tarefas, contratos, dados, riscos e reversibilidade na spec. Depois, transforma os cenários BDD em testes TDD executáveis e observa o **RED**: uma falha causada pela funcionalidade que ainda não existe.

O ato termina com:

```
Plan Gate: Passed
```

O RED precisa falhar pelo motivo esperado. Erro de sintaxe, dependência ausente ou ambiente quebrado não prova que o teste protege o comportamento.

Ato III — Entregar e conferir o resultado

Objetivo: implementar cada tarefa e reunir evidências atuais de que a entrega atende à definição.

Sua participação: você acompanha decisões novas ou mudanças de escopo. Não precisa comandar cada ciclo de teste; o agente registra o que foi executado e apresenta o resultado verificável.

Prova técnica: cada tarefa segue:

```
RED → GREEN → REFACTOR
```

- **RED:** o teste falha pela razão esperada;
- **GREEN:** a menor implementação faz o teste passar;
- **REFACTOR:** o código é melhorado sem mudar o comportamento.

Depois, o agente executa o aceite, a regressão completa, verifica a rastreabilidade entre requisito e teste, atualiza o contexto e reconstrói a documentação aplicável.

O ato termina com:

```
Delivery Gate: Passed  
Status: Complete
```

Complete significa que a entrega possui código e evidência atual. Não é apenas uma tarefa marcada como concluída.

Como BDD e TDD trabalham juntos

BDD e TDD têm papéis diferentes e complementares.

O **BDD** descreve o comportamento em uma linguagem que produto, desenvolvimento e testes conseguem discutir. Por exemplo:

```
Cenário: pessoa solicita recuperação de senha
  Dado que existe uma conta para o e-mail informado
  Quando a pessoa solicita a recuperação
  Então o sistema confirma o pedido sem revelar dados privados
```

Esse cenário deixa visível a regra e o resultado esperado, mas o texto Gherkin não é executado como uma suíte separada pelo Specsify.

O **TDD** transforma o comportamento em testes executáveis na ferramenta já usada pelo projeto. Primeiro o teste falha pelo motivo correto; depois a implementação faz o teste passar. Em resumo:

- BDD ajuda a definir **o comportamento que importa**;
- TDD comprova **que o código apresenta esse comportamento**.

Essa ligação evita dois extremos: uma descrição clara sem prova automática, ou um teste técnico que não representa o que foi pedido.

O agente conduz as transições

As skills dividem responsabilidades, mas você não precisa operar o fluxo como uma lista de comandos. Quando uma etapa depende de outra, o agente:

1. informa de qual skill está saindo e para qual está indo;
2. explica a pendência que motivou a transição;
3. resolve a pendência no contexto correto;
4. retorna à etapa anterior quando necessário.

Por exemplo, se a implementação encontrar um teste ausente, o agente volta ao planejamento e à preparação TDD, obtém um RED válido e só então retoma o código. Ele não aprova um gate apenas para contornar a pendência.

Mudanças durante o trabalho

Se o pedido mudar depois que a spec já existe, você pode explicar a alteração em linguagem normal. A mudança entra na mesma `spec.md`; não é criada uma segunda especificação.

O Specsify reabre somente as provas que perderam validade:

- se mudou o comportamento, reabre definição, plano e entrega;
- se mudou somente a estratégia técnica, reabre plano e entrega;
- se apenas uma evidência ficou desatualizada, repete a validação necessária.

Use [specsify-base-update-spec](#) para esse fluxo.

Contexto que permanece entre entregas

Além da spec de cada entrega, o projeto mantém informações que valem para o sistema inteiro:

- `PROJECT.md` : finalidade, capacidades e limites do projeto;
- `.specsify/STACK.md` : tecnologias estruturais e suas evidências;
- `.specsify/RULES.md` : regras confirmadas para o trabalho;
- `.specsify/DATABASE.md` : mapa da persistência e das relações.

O agente consulta esse contexto antes de planejar e o revisa durante a implementação. Assim, uma nova entrega não precisa redescobrir a arquitetura, as convenções ou o banco desde o início.

O que você encontra ao final

Uma entrega completa deixa um caminho auditável:

- a intenção e as decisões estão na spec;
- cada requisito aponta para critérios de aceite;
- os cenários BDD explicam o comportamento;
- os testes TDD demonstram o resultado no código;
- as tarefas registram execução e evidências;
- os gates mostram quais etapas foram realmente comprovadas;
- a documentação reflete o sistema implementado.

Para consultar esse estado sem alterar arquivos, use [specsify-base-progress](#).

Limites do método

O método não decide requisitos importantes sem você, não transforma toda ideia em spec e não trata pesquisa como requisito aprovado. Também não aceita erro de ambiente como RED nem substitui os testes e as ferramentas do seu projeto.

Agora siga a [instalação](#) e faça o [primeiro projeto](#).

Instalação do Specsify

1. Instale o CLI

O CLI requer Python 3.11 ou superior. Baixe o executável diretamente de `get.specsify.dev` para a pasta de comandos do seu usuário:

```
mkdir -p "$HOME/.local/bin"
curl -fL get.specsify.dev -o "$HOME/.local/bin/specsify"
chmod +x "$HOME/.local/bin/specsify"
```

Confira a instalação:

```
specsify --version
```

Se o terminal ainda não encontrar o comando, inclua a pasta no `PATH` da sessão e tente novamente:

```
export PATH="$HOME/.local/bin:$PATH"
specsify --version
```

Para atualizar o CLI depois, repita o download. O novo arquivo substitui o executável anterior:

```
curl -fL get.specsify.dev -o "$HOME/.local/bin/specsify"
chmod +x "$HOME/.local/bin/specsify"
```

2. Prepare seu projeto

Entre no projeto em que você quer usar o Specsify e execute:

```
cd caminho/do/projeto
specsify install --project .
```

O comando prepara o projeto para trabalhar com a metodologia:

- instala o fluxo base em `.agents/skills/specsify-base-*` ;
- adiciona setup, documentação e contexto para o agente entender o projeto;
- publica o contrato central em `.specsify/Spec.md` ;
- adiciona templates, exemplos e registros técnicos dentro de `.specsify/` ;
- integra orientações gerenciadas a `AGENTS.md` e `CLAUDE.md` sem apagar seu conteúdo.

Essa etapa não cria uma especificação de produto. Ela apenas deixa o ambiente pronto para você iniciar a primeira entrega.

3. Confirme que está pronto

Na raiz do projeto, execute:

```
specsfy skills list  
specsfy progress --project .
```

O primeiro comando deve listar as skills instaladas. O segundo pode informar zero specs; isso é esperado em um projeto novo e confirma que o CLI consegue ler o ambiente.

Você também pode abrir a interface visual:

```
specsfy
```

Se algo não funcionar

- **specsfy: command not found**: adicione `$HOME/.local/bin` ao `PATH` e abra um novo terminal.
- **Python incompatível**: confirme com `python3 --version`; o CLI requer Python 3.11 ou superior.
- **Falha ao instalar skills**: disponibilize o comando `skills` ou o `npx`, usado como alternativa pelo CLI.
- **Arquivo local protegido**: uma instalação repetida preserva customizações. Revise a diferença antes de usar `--force`, pois essa opção descarta o conteúdo protegido.

Próximo passo

Siga o [primeiro projeto](#) para transformar um pedido em uma entrega validada. Depois, consulte o [guia do CLI e da TUI](#) quando precisar de comandos, progresso ou opções avançadas.

Uso básico do Specsify

Papel

Conduzir um texto capturado até uma entrega comprovada, mantendo uma única fonte normativa e sem exigir que a pessoa memorize toda a sequência de skills.

Como usar

Pré-condições

- CLI e framework instalados conforme o [guia de instalação](#);
- agente aberto na raiz do projeto consumidor;
- uma ideia de produto ou mudança que possa ser descrita em linguagem comum.

Veja o Specsify trabalhando

Vamos criar uma página de boas-vindas em um projeto Laravel que já usa Pest. Os dez comandos abaixo mostram a jornada completa.

1. Capture a ideia — `$specsify-base-idea`

```
Use $specsify-base-idea para capturar:  
criar uma página /boas-vindas que cumprimente a pessoa pelo nome.
```

Resultado, sem perguntas:

```
Ideia capturada em  
specs/ideias/2026-07-28-143205-pagina-boas-vindas.md
```

2. Refine no backlog — `$specsify-base-backlog`

```
Use $specsify-base-backlog para guardar esta ideia:  
criar uma página /boas-vindas que cumprimente a pessoa pelo nome.
```

Também pode usar a captura:

```
Use $specsify-base-backlog para refinar  
specs/ideias/2026-07-28-143205-pagina-boas-vindas.md
```

Resultado:

```
Ideia registrada em specs/backlog/0001-pagina-boas-vindas.md
```

3. Tire as dúvidas — \$specsfy-base-interview

Opção 1 — texto livre

```
Use $specsfy-base-interview para aprofundar este texto:  
quero uma página /boas-vindas que cumprimente a pessoa pelo nome.
```

Opção 2 — arquivo de backlog

```
Use $specsfy-base-interview em specs/backlog/0001-pagina-boas-vindas.md
```

O agente pergunta somente o que realmente falta:

```
Agente: O que deve aparecer quando nenhum nome for informado?  
Você: Olá, visitante!  
  
Brief pronto para especificar.
```

4. Crie a especificação — \$specsfy-base-specify

Opção 1 — texto livre

```
Use $specsfy-base-specify para criar uma especificação a partir deste texto:  
a página /boas-vindas mostra Olá e o nome informado; sem nome, usa visitante.
```

Opção 2 — arquivo de backlog

```
Use $specsfy-base-specify para promover specs/backlog/0001-pagina-boas-vindas.md
```

Resultado:

```
Especificação criada em  
specs/specs/0001-pagina-boas-vindas/spec.md  
3 cenários BDD cobrem a feature, sua história e seus requisitos.
```

5. Confira a especificação — `$specsfy-base-validate`

```
Use $specsfy-base-validate em specs/specs/0001-pagina-boas-vindas/spec.md
```

Resultado:

```
READY  
Definition Gate: Passed
```

`READY` significa que o pedido está claro o bastante para seguir.

6. Divida o trabalho — `$specsfy-base-tasks`

```
Use $specsfy-base-tasks em specs/specs/0001-pagina-boas-vindas/spec.md
```

Resultado:

```
2 tarefas preparadas.
```

7. Prepare a verificação — `$specsfy-base-tdd-bdd`

```
Use $specsfy-base-tdd-bdd em specs/specs/0001-pagina-boas-vindas/spec.md  
para preparar a verificação.
```

Resultado:

```
Verificação preparada: 3 casos TDD com marcadores SPECSFY: próprios.  
RED observado antes da implementação.
```

O mínimo de três usa contextos diferentes — por exemplo, caminho feliz, variação crítica e falha material — sem duplicar o mesmo exemplo.

8. Implemente — `$specsfy-base-implement`

```
Use $specsfy-base-implement em specs/specs/0001-pagina-boas-vindas/spec.md
```

Resultado:

```
Implementação concluída.  
Página /boas-vindas criada.  
Verificação aprovada.
```

9. Altere a especificação — `$specsfy-base-update-spec`

Depois de implementar, imagine que você lembrou de uma regra:

```
Use $specsfy-base-update-spec em  
specs/specs/0001-pagina-boas-vindas/spec.md:  
o nome deve ter no máximo 80 caracteres.
```

Resultado:

```
Pedido incorporado na especificação 0001-pagina-boas-vindas.  
Etapas afetadas retomadas automaticamente.  
Implementação atualizada.
```

A mudança continua na mesma spec e volta apenas às etapas necessárias.

10. Veja o progresso — `$specsfy-base-progress`

```
Use $specsfy-base-progress para mostrar o resultado final.
```

Resultado:

```
Complete · 3/3 etapas · nenhuma pendência
```

Você pode conferir a mesma entrega pelo CLI:

```
specsfy progress --project .  
specsfy progress --project . --json  
specsfy tui --project .
```

Ao autorizar a jornada completa, você não precisa enviar os dez comandos manualmente: cada skill pode chamar a próxima e continuar na mesma conversa. O passo a passo separado serve para aprender, inspecionar ou retomar qualquer etapa.

Resultado esperado

Ao final, a ideia está registrada, as dúvidas foram resolvidas, a especificação foi validada, a página foi implementada e o progresso mostra `Complete`.

Limites

- `specsify-base-update-spec` faz mudança de comportamento reabrir os Atos I–III;
- `specsify-base-update-spec` faz mudança de plano reabrir os Atos II–III;
- backlog não autoriza implementação;
- não crie `plan.md`, `tasks.md`, `research.md` ou `data-model.md`;
- deploy, publicação, instalação e ações destrutivas mantêm autorização própria.

Próximos passos

- [Uso avançado](#)
- [CLI e TUI](#)
- [Contexto persistente do projeto](#)
- [Documentação técnica do sistema](#)

Atualize quando

- a sequência dos atos ou a responsabilidade de uma skill base mudar;
- os gates, estados ou caminhos canônicos mudarem;
- a primeira jornada do usuário ganhar ou perder uma etapa.

Não use para

- definir comandos avançados do CLI;
- reproduzir o contrato completo de cada skill;
- registrar requisitos ou evidências de um projeto consumidor.

Fonte da verdade e precedência

A metodologia executável pertence a `skills/`. A fonte normativa de cada fatia pertence ao projeto consumidor em `specs/specs/<NNNN>-<slug>/spec.md`.

Caixa de entrada de ideias

`specs/ideias/` recebe pensamentos antes de qualquer entrevista ou decisão de produto. A captura é deliberadamente rápida: o agente não faz perguntas e não transforma o texto em compromisso de entrega.

Capturar

Use `$specsfy-base-idea` para guardar esta ideia:
quero permitir que a pessoa continue um formulário em outro dispositivo.

O resultado usa o formato:

`specs/ideias/AAAA-MM-DD-HHMMSS-<slug>.md`

Data e hora evitam uma fila numerada artificial e preservam a ordem real de chegada. Se duas capturas tiverem o mesmo nome no mesmo segundo, o Specsify adiciona um sufixo sem sobrescrever a anterior.

O que o arquivo organiza

- metadados e integridade do texto original;
- texto original completo;
- resumo processado;
- problema ou oportunidade;
- pessoas e valor percebidos;
- sinais de escopo, regras ou solução;
- riscos, dependências e direções possíveis;
- pontos a revisar no futuro;
- rastreabilidade para backlog ou spec derivados.

Declarações, inferências e lacunas permanecem identificadas. Campo sem base no texto é marcado como não identificado; o agente não inventa respostas.

O texto será versionado no Git. Não inclua senhas, tokens, chaves privadas ou dados pessoais sensíveis. Se o agente detectar um segredo evidente, ele não grava a captura e pede apenas que o conteúdo sensível seja removido.

Ideia, backlog e spec

captura sem perguntas → backlog refinável → spec normativa

- `specs/ideias/` preserva o input;
- `specs/backlog/` organiza algo escolhido para refinamento;
- `specs/specs/<NNNN>-<slug>/spec.md` governa comportamento e entrega.

Uma captura pode permanecer indefinidamente na caixa de entrada. Quando quiser avançar, use `$specsfy-base-backlog` ou `$specsfy-base-interview` com o caminho do arquivo.

Templates instalados

O CLI mantém os templates documentais em `.specsfy/templates/`:

```
Idea.md
Backlog.md
Spec.md
Tasks.md
Project.md
Stack.md
Rules.md
Database.md
```

Alterações locais são protegidas pelo instalador e só são substituídas com `--force`.

Backlog de requisitos e promoção de ideias

Papel

Explicar como transformar capturas e necessidades ainda abertas em um backlog organizado, priorizado e verificável antes da promoção.

Como usar

Consulte ao registrar uma ideia, escolher entre backlog e entrevista ou promover um item para spec. Execute as skills pelos nomes indicados no fluxo.

Atualize quando

- a estrutura de backlog ou specs mudar;
- uma skill assumir outra etapa do fluxo;
- estados ou critérios de promoção mudarem.

Não use para

- substituir os requisitos formais e gates da spec promovida;
- substituir a spec promovida;
- autorizar implementação diretamente do backlog.

Fonte da verdade e precedência

Os arquivos do projeto preservam o estado; as skills em `skills/` governam o comportamento executável. Depois da promoção, a spec prevalece sobre o backlog para comportamento, gates, tarefas e evidência. Para preservar sem perguntas, use a [caixa de entrada de ideias](#).

Estrutura

```
specs/  
├─ ideias/  
│   └─ 2026-07-28-143205-ideia.md  
├─ backlog/  
│   └─ 0001-ideia.md  
└─ specs/  
    └─ 0001-feature/  
        ├── spec.md  
        └─ research/
```

O item de backlog começa leve, mas pode amadurecer até produto, desenvolvimento e testes compreenderem o comportamento esperado. Ele não contém tarefas ou gates e nunca autoriza implementação diretamente.

Organização do backlog

```
Produto  
└─ Épico  
    └─ Funcionalidade  
        ├── História ou requisito  
        ├── Regra  
        ├── Item técnico  
        └─ Melhoria
```

O épico representa um objetivo amplo. A funcionalidade delimita uma capacidade. Histórias e requisitos representam entregas menores e verificáveis. Regras, itens técnicos e melhorias também pertencem ao backlog quando tornam o comportamento ou sua operação possível.

Nem toda ideia precisa nascer com a hierarquia completa. Use **A esclarecer** no lugar de inventar uma relação.

Anatomia de um item

Logo abaixo do título, mantenha as metainformações em uma tabela, não em lista:

METAINFORMAÇÃO	VALOR
ID	BACKLOG-0001
Status	Captured

METAINFORMAÇÃO	VALOR
Produto	A esclarecer
Épico	A esclarecer
Funcionalidade	A esclarecer
Tipo	A esclarecer
Prioridade	Não priorizado
Criado em	data ISO
Spec promovida	Nenhuma

Um item bem refinado pode conter:

- título, tipo e prioridade;
- produto, épico e funcionalidade;
- contexto do problema;
- objetivo ou história do usuário;
- comportamento esperado;
- regras de negócio;
- critérios de aceitação;
- segurança, privacidade, desempenho, volume e auditoria;
- dependências;
- situações de erro e exceções;
- dentro e fora de escopo.

A profundidade é adaptativa. Uma alteração simples exige menos detalhes; autenticação, pagamentos, permissões, privacidade e processamento assíncrono exigem análise cuidadosa. O melhor item não é o mais longo: é o que reduz ambiguidade e permite verificar a entrega.

Captura mínima e conversa

Antes de escrever, `$specsfy-base-backlog` reaproveita o pedido e confirma:

- problema percebido;
- pessoa afetada ou beneficiada;
- resultado ou valor esperado;
- contexto suficiente para distinguir a ideia.

Se algo estiver ausente, vago, contraditório ou ambíguo, a skill pergunta uma lacuna relevante por vez e reavalia após cada resposta. Ela não usa questionário fixo, não repete o que já foi explicado e não persiste placeholders nesses campos. Se a pessoa não souber responder, explicita a lacuna sem inventar.

Esse é o mínimo de captura, não uma entrevista profunda. Hierarquia, prioridade, regras detalhadas, aceite e solução técnica podem ser refinados depois.

Duplicatas e referências

Antes de criar, a skill pesquisa termos do pedido em `specs/backlog/*.md`, `specs/specs/*/spec.md` e `docs/**/*.md`. Ela separa possível duplicata de backlog relacionado, spec relacionada ou documentação útil.

Uma possível duplicata exige confirmar se o item existente será atualizado ou se há uma diferença real. Fontes úteis ficam em `Referências relacionadas` com caminho e relação, sem substituir a intenção declarada pelo usuário.

Padrões recorrentes

CAPACIDADE	QUESTÕES QUE O BACKLOG DEVE TORNAR OBJETIVAS
Autenticação	estado da conta, tentativas, sessão, mensagens seguras, credenciais e autorização
Notificações	propriedade, contagem, evento de leitura, ações individual/em lote e isolamento
Permissões	perfil, operação, organização, alvo, invariantes, validação no servidor e auditoria
PIX/pagamentos	fluxo, estados, idempotência, webhook, falhas externas e dados sensíveis
Exportação	filtros, autorização, volume, fila, expiração, fuso horário e dados sensíveis

Use a representação que reduz ambiguidade. Fluxos numerados ajudam em integrações; matrizes ajudam em permissões; cenários ajudam a demonstrar resultados e erros.

Um requisito funcional descreve o que o sistema faz. Um requisito não funcional define condições mensuráveis de qualidade e operação. Troque “o sistema deve ser rápido” por um limite de latência, volume e ambiente verificáveis.

Priorização

Mantenha o backlog realmente ordenado. Compare:

1. valor para usuário e negócio;
2. risco de segurança, privacidade e operação;
3. dependências e entregas desbloqueadas;
4. urgência;
5. esforço;
6. incerteza.

Prioridade é relativa; classificar tudo como alta não cria uma ordem. Autenticação e autorização podem preceder melhorias visuais, assim como uma infraestrutura de notificações pode preceder os eventos que dependem dela.

Exemplo de visão ordenada:

ORDEM	PRIORIDADE	TIPO	ITEM	OBJETIVO
1	Alta	Épico	Gestão de usuários	Administrar acesso
2	Alta	História	Realizar login	Acessar áreas privadas
3	Alta	Regra	Controlar permissões	Restringir operações por perfil
4	Média	Técnico	Notificações em fila	Evitar lentidão nas requisições
5	Baixa	Melhoria	Preferências de notificação	Escolher canais

Cada linha aponta para um item próprio. Por exemplo, “realizar login” deve esclarecer conta ativa, comparação de e-mail, proteção da senha, tentativas inválidas, sessão, permissões, resultados de sucesso e falha e o que ficou fora do escopo. “Criar uma tela de login” não cobre esse comportamento.

Fluxo

```
input → ideia capturada → backlog → interview → spec
```

1. Use `$specsify-base-backlog` quando a ideia ainda for geral. A skill pesquisa material relacionado, conversa até a captura mínima ficar clara e produz `specs/backlog/<NNNN>-<slug>.md`. A mesma skill pode organizar, priorizar e refinar o item progressivamente.
2. Use `$specsify-base-interview` para aprofundar uma ideia, um backlog ou uma spec. A entrevista faz uma pergunta relevante por vez e produz um brief.

3. Use `$specsfy-base-specify` somente quando houver intenção explícita de promover o material. A fonte normativa nasce em `specs/specs/<NNNN>-<slug>/spec.md`.
4. Depois da promoção, mantenha o backlog como proveniência, marque-o `Promoted` e registre o caminho da spec.

Ao concluir, a skill anuncia e executa automaticamente o avanço ou retorno; a conversa continua sem repetir comandos nem autorizar implementação.

Backlog e spec possuem sequências independentes. `BACKLOG-0004` pode originar `SPEC-0002`; identidade não implica prioridade.

Estados do backlog

ESTADO	SIGNIFICADO
Captured	ideia preservada com contexto mínimo
Refining	conversa ou pesquisa leve em andamento
Ready for interview	contexto suficiente para aprofundamento estruturado
Promoted	spec derivada criada e referenciada

Quando está refinado

Antes do handoff, a equipe deve conseguir responder:

- Qual problema será resolvido e quem será beneficiado?
- Qual evento inicia o comportamento e qual resultado será produzido?
- Quem pode executar a operação?
- Quais regras, erros e exceções precisam ser respeitados?
- Como verificar objetivamente o resultado?
- Há implicações de segurança, privacidade, desempenho ou volume?
- O que ficou fora da entrega?
- Quais dependências ou decisões continuam pendentes?

O agente identifica lacunas e pergunta uma por vez. Decisões que alteram segurança, escopo, arquitetura ou experiência não são inventadas silenciosamente. Esse diagnóstico prepara entrevista e spec; ainda não autoriza desenvolvimento.

Limites

- Não implementar diretamente de um backlog.
- Não exigir arquitetura, Gherkin ou plano técnico durante a captura inicial.

- Não apagar a formulação original ao resumir.
- Não promover automaticamente uma ideia.
- Não tratar o backlog como fonte normativa depois da promoção.

Implementação executável

O contrato executável pertence às skills `specsfy-base-backlog`, `specsfy-base-interview` e `specsfy-base-specify`.

Justificativa de tamanho

O guia reúne estrutura, refinamento, priorização e promoção porque essas decisões precisam ser comparadas no mesmo percurso público.

Skills base



As skills base dividem o método em responsabilidades pequenas. Você pode pedir uma delas pelo nome ou simplesmente explicar o que quer; o agente seleciona e encadeia as etapas necessárias.

ETAPA	SKILL	RESULTADO PRINCIPAL
capturar sem perguntas	specsfy-base-idea	arquivo em <code>specs/ideias/</code>
guardar uma ideia	specsfy-base-backlog	item em <code>specs/backlog/</code>
aprofundar decisões	specsfy-base-interview	brief na conversa
criar a fonte única	specsfy-base-specify	<code>spec.md</code>
revisar definição	specsfy-base-validate	Definition Gate confiável
decompor o plano	specsfy-base-tasks	tarefas dentro da spec
preparar e executar testes	specsfy-base-tdd-bdd	RED/GREEN rastreável
produzir a mudança	specsfy-base-implement	código, testes e evidência
incorporar pedido tardio	specsfy-base-update-spec	spec atualizada e gates reabertos
consultar o estado	specsfy-base-progress	relatório somente leitura

Como escolher

- Quer apenas guardar o texto agora? Comece pela captura de ideia.
- Quer refinar uma captura vaga? Use o backlog.
- Você quer decidir detalhes antes de criar uma spec? Use a entrevista.
- A entrega está clara, mas ainda não possui `spec.md` ? Use specify.
- Quer saber se a definição está pronta? Use validate.
- A definição está aprovada e precisa virar passos executáveis? Use tasks.
- Precisa criar ou comprovar testes? Use TDD/BDD.
- O plano está aprovado e há tarefa pronta? Use implement.
- O pedido mudou depois da definição? Use update-spec.
- Quer apenas saber quanto falta? Use progress.

Volte ao [guia completo](#) ou leia [como a metodologia funciona](#).

Capturar uma ideia com `specsfy-base-idea`

Esta skill funciona como uma caixa de entrada: recebe seu texto, guarda o original e organiza uma primeira leitura sem interromper você com perguntas.

Quando usar

Use quando quiser anotar uma ideia, oportunidade, necessidade ou pensamento para retomar depois. O arquivo fica em `specs/ideias/`.

Não use para refinar prioridade, decidir requisitos ou iniciar implementação. Esses trabalhos pertencem às etapas seguintes.

Como pedir

```
Use $specsfy-base-idea para capturar:  
seria útil avisar clientes quando uma entrega atrasar, talvez por e-mail.
```

Você também pode simplesmente pedir “guarde esta ideia” e incluir o texto.

Exemplo passo a passo

1. Você envia o texto livre.
2. O agente preserva o texto original.
3. Sem fazer perguntas, ele extrai resumo, problema ou oportunidade, pessoas, valor esperado, sinais, riscos e pontos a revisar.
4. Ele cria um nome com data, hora e slug.
5. Ele informa o caminho criado e encerra a captura.

O que esperar

```
specs/ideias/2026-07-28-143205-avisar-clientes-sobre-atrasos.md
```

O arquivo diferencia o que você declarou, o que foi inferido e o que ainda precisa ser revisto. Nenhum backlog, spec, tarefa ou código é criado.

Os metadados incluem horário da captura, origem, slug, status, hash do texto original e links futuros para backlog ou spec.

Erros comuns

- esperar que a captura faça perguntas ou complete decisões ausentes;
- tratar a análise inicial como requisito aprovado;
- implementar diretamente a partir de `specs/ideias/` ;
- editar o texto original em vez de registrar uma evolução no backlog;
- procurar o template dentro da skill: ele vive em `.specsfy/templates/Idea.md` .

Próximo passo

Deixe a ideia guardada ou use [specsfy-base-backlog](#) quando quiser refiná-la.

Guardar uma ideia com `specsfy-base-backlog`

Esta skill refina uma ideia sem exigir todas as respostas agora. Ela conversa de forma leve, procura itens parecidos e cria ou atualiza um arquivo em `specs/backlog/`.

Quando usar

Use quando quiser organizar uma captura em `specs/ideias/`, trazer um problema ainda vago ou quiser avaliar uma oportunidade antes de criar uma especificação. Para apenas salvar sem perguntas, use `specsfy-base-idea`.

Não use para planejar tarefas, escrever testes ou iniciar código.

Como pedir

Você pode escrever naturalmente:

```
Use $specsfy-base-backlog para guardar esta ideia:
permitir que a pessoa escolha o idioma da interface.
```

Também pode informar contexto:

```
Anote no backlog: clientes internacionais não entendem os e-mails atuais.
Ainda não decidimos quais idiomas serão oferecidos.
```

Exemplo passo a passo

1. Você apresenta a ideia.
2. O agente também pode ler a origem em `specs/ideias/`.
3. Ele procura itens semelhantes em `specs/backlog/` e nas specs.
4. Ele pergunta somente o necessário para diferenciar a ideia.
5. Você responde: “o problema afeta e-mails e a interface”.
6. A skill cria o item com `.specsfy/templates/Backlog.md`:

```
specs/backlog/0003-idioma-da-interface.md
```

O item registra problema, público, resultado esperado e dúvidas abertas. Ele continua sendo uma ideia; não autoriza implementação.

O que esperar

- poucas perguntas;
- preservação das suas palavras;
- indicação de duplicatas ou relações;
- um caminho claro para retomar depois;
- nenhuma `spec.md` criada automaticamente sem promoção.

Erros comuns

- transformar o backlog em uma especificação completa;
- inventar solução técnica quando o problema ainda está aberto;
- criar um segundo item sem procurar duplicatas;
- tratar o item como aprovação para implementar.

Próximo passo

Quando quiser aprofundar a ideia, use [specsfy-base-interview](#). Se ela já estiver clara, você pode promovê-la com [specsfy-base-specify](#).

Aprofundar uma ideia com `specsfy-base-interview`

Esta skill conduz uma conversa para transformar dúvidas importantes em decisões testáveis. Ela faz uma pergunta prioritária por vez e entrega um brief na própria conversa.

Quando usar

Use quando uma ideia ou backlog precisa de mais clareza: público, finalidade, regras, limites, privacidade, falhas ou resultado esperado.

Não use para escrever `spec.md`; essa responsabilidade é da skill `specify`.

Como pedir

Com um arquivo de backlog:

```
Use $specsfy-base-interview para aprofundar
specs/backlog/0003-idioma-da-interface.md.
```

Com texto livre:

```
Use $specsfy-base-interview para me ajudar a definir uma recuperação de senha.
```

Exemplo passo a passo

1. O agente resume: “pessoas sem acesso à senha precisam recuperar a conta”.
2. Ele pergunta: “como a pessoa comprova que controla a conta?”.
3. Você responde: “por um link enviado ao e-mail cadastrado”.
4. Ele pergunta sobre expiração, mensagens de erro e privacidade.
5. Ao terminar, entrega:

```
Brief pronto para especificar:
- ator: pessoa com conta existente;
- resultado: receber um link temporário;
- limite: não revelar se o e-mail existe;
- aberto: duração do link.
```

Se uma decisão material continuar aberta, a entrevista não finge que a definição está pronta.

O que esperar

- perguntas adaptadas ao seu caso;
- uma lacuna importante por vez;
- distinção entre fato, hipótese e decisão;
- exemplos de sucesso e falha;
- um brief pronto para a próxima etapa.

Erros comuns

- responder todas as categorias como um formulário fixo;
- decidir algo importante no lugar da pessoa responsável;
- esconder uma dúvida para acelerar;
- criar arquivos normativos durante a entrevista.

Próximo passo

Use [specsfy-base-specify](#) para criar a especificação. Se a conversa mostrar que a ideia ainda é superficial, volte ao [specsfy-base-backlog](#).

Criar a especificação com `specsfy-base-specify`

Esta skill cria a fonte única da entrega. Ela reúne descoberta, requisitos, comportamentos, plano, testes, tarefas e evidências no mesmo `spec.md`.

Quando usar

Use para promover um backlog entrevistado ou criar uma spec nova ainda em estado Draft. Para mudar uma spec que já foi definida, use `update-spec`.

Como pedir

Com um backlog:

```
Use $specsfy-base-specify para promover
specs/backlog/0003-idioma-da-interface.md.
```

Com um brief na conversa:

```
Use $specsfy-base-specify para criar uma especificação para recuperação de senha
com base nas decisões desta conversa.
```

Exemplo passo a passo

1. A skill confirma a raiz do projeto e o próximo número disponível.
2. Ela lê contexto, instruções e evidências necessárias.
3. Cria o pacote:

```
specs/specs/0004-recuperar-senha/
├── spec.md
└── research/          # somente quando houver pesquisa externa
```

1. Preenche o Ato I com requisitos e cenários.
2. Mantém decisões ainda abertas claramente marcadas.
3. Executa validadores sem inventar uma aprovação.

Research apoia decisões, mas nunca substitui o texto normativo de `spec.md`.

O que esperar

- formato Specsify/2.0 ;
- IDs rastreáveis para histórias, requisitos e critérios;
- cenários que cobrem sucesso, variação e falha;
- uma única fonte normativa;
- status Draft ou Defined conforme a evidência real.

Erros comuns

- criar `plan.md`, `tasks.md` ou `research.md` ;
- copiar uma fonte externa como requisito sem decisão;
- aprovar gates com campos incompletos;
- misturar várias entregas grandes na mesma spec.

Próximo passo

Use [specsify-base-validate](#) para provar que a definição está pronta. Se uma decisão importante faltar, a transição volta para [specsify-base-interview](#) .

Validar a definição com `specsify-base-validate`

Esta skill revisa a spec como código em linguagem natural. Primeiro verifica o formato; depois avalia clareza, completude, consistência e possibilidade de teste.

Quando usar

Use antes de planejar tarefas ou quando uma mudança reabrir o Ato I. Também é útil para auditar uma spec sem alterar sua intenção.

Como pedir

```
Use $specsify-base-validate em
specs/specs/0004-recuperar-senha/spec.md.
```

Exemplo passo a passo

1. A skill confirma o caminho e o formato `Specsfy/2.0`.
2. Verifica se os requisitos possuem exemplos suficientes.
3. Encontra uma lacuna: “a validade do link não foi decidida”.
4. Retorna para a entrevista e registra a decisão de 30 minutos.
5. Executa novamente os validadores.
6. Atualiza a seção de gates:

```
Definition Gate: Passed
Status: Defined
```

Uma falha de parser, fixture ou ambiente não conta como prova de requisito ausente.

O que esperar

- problemas apontados com localização e motivo;
- nenhuma aprovação baseada em suposição;
- verificação das evidências externas citadas;
- retorno automático à etapa que pode resolver a lacuna;
- gate aprovado somente após nova validação.

Erros comuns

- marcar READY sem resolver uma ambiguidade material;
- confundir estrutura válida com conteúdo suficiente;
- criar um relatório paralelo em vez de atualizar a seção correta;
- compensar um Ato I incompleto em uma etapa posterior.

Próximo passo

Com o Definition Gate aprovado, use [specsify-base-tasks](#) . Se a validação encontrar uma decisão aberta, use [specsify-base-interview](#) .

Planejar tarefas com `specsify-base-tasks`

Esta skill transforma uma definição aprovada em tarefas pequenas, ordenadas e verificáveis. As tarefas ficam na seção 14 da própria `spec.md`.

Quando usar

Use depois do Definition Gate ou quando uma alteração exigir replanejamento. Não use para escrever código nem para marcar uma tarefa como concluída.

Como pedir

```
Use $specsify-base-tasks em
specs/specs/0004-recuperar-senha/spec.md.
```

Você também pode pedir uma fatia:

```
Prepare as tarefas da primeira fatia vertical da spec 0004.
```

Exemplo passo a passo

1. A skill lê a spec e o código existente.
2. Identifica a menor entrega observável: solicitar o link.
3. Liga cada tarefa aos requisitos e critérios correspondentes.
4. Declara predecessoras de teste antes da tarefa de produção.
5. Registra:

```
T001 [ ] Criar caso TDD para solicitação válida – cobre AC-001
T002 [ ] Criar caso TDD para e-mail desconhecido – cobre AC-002
T003 [ ] Implementar solicitação sem revelar existência da conta
```

1. Confirma que dependências, rollback e validações estão claros antes de aprovar o Plan Gate.

O que esperar

- tarefas pequenas e com resultado verificável;
- ordem explícita de dependência;
- testes antes de produção;
- caminhos e comandos reais do projeto;

- tarefas mantidas dentro da fonte única.

Erros comuns

- criar `tasks.md` ;
- escrever tarefas vagas como “fazer backend”;
- colocar várias mudanças independentes em uma tarefa;
- planejar sem inspecionar a stack real;
- marcar uma tarefa pronta sem evidência.

Próximo passo

Use [specsify-base-tdd-bdd](#) em modo `prepare` para materializar o próximo teste e observar RED.

Preparar testes com `specsify-base-tdd-bdd`

Esta skill usa os cenários da spec para criar testes executáveis. Ela mantém a rastreabilidade entre o comportamento descrito e a prova no código.

Quando usar

Use para preparar o RED antes da implementação, executar um ciclo RED–GREEN–REFACTOR ou verificar testes e rastreabilidade.

Como pedir

Antes do código:

```
Use $specsify-base-tdd-bdd em modo prepare para
specs/specs/0004-recuperar-senha/spec.md.
```

Para verificar uma entrega existente:

```
Use $specsify-base-tdd-bdd em modo verify na spec 0004.
```

Exemplo passo a passo

1. A skill seleciona o próximo critério de aceite.
2. Encontra o runner real do projeto.
3. Cria um teste com marcador de rastreabilidade.
4. Executa somente o teste focal.
5. Confirma:

```
RED válido: o teste falhou porque o pedido de recuperação ainda não existe.
Caso: TDD-AC-001
```

1. Depois da implementação, repete o teste e a regressão.

O arquivo Gherkin da spec é uma referência legível. A prova automatizada fica na suíte normal do projeto, não em uma segunda suíte paralela.

O que esperar

- caso de teste ligado a um critério da spec;

- comando e resultado registrados;
- distinção entre falha esperada e problema de ambiente;
- cobertura de sucesso, regra e limite;
- regressão depois do GREEN.

Erros comuns

- chamar erro de configuração de RED;
- escrever produção no modo `prepare` ;
- criar testes que não correspondem aos critérios;
- considerar o Gherkin sozinho como teste executado;
- aprovar o plano sem prova focal.

Próximo passo

Com RED válido e tarefa pronta, use [specsfy-base-implement](#) . Para apenas reorganizar as tarefas, volte a [specsfy-base-tasks](#) .

Entregar código com `specsfy-base-implement`

Esta skill executa as tarefas aprovadas da spec em ordem. Ela altera produção somente quando existe um plano válido e um teste focal em RED.

Quando usar

Use para implementar a próxima tarefa pronta, continuar uma entrega ou concluir uma feature planejada. Não use para pular definição, planejamento ou testes.

Como pedir

Use `$specsfy-base-implement` para executar a próxima tarefa pronta de `specs/specs/0004-recuperar-senha/spec.md`.

Ou indique uma tarefa:

Implemente T003 da spec 0004 e valide a regressão.

Exemplo passo a passo

1. A skill confirma Definition Gate e Plan Gate aprovados.
2. Verifica a tarefa predecessora e o RED atual.
3. Faz a menor mudança de produção.
4. Executa o teste focal até obter GREEN.
5. Refatora sem alterar o comportamento.
6. Executa a regressão e atualiza evidências:

T003 [x] Implementar solicitação sem revelar existência da conta
Teste focal: passou
Regressão: passou

1. Ao concluir a entrega, chama o documentador do projeto consumidor.

O que esperar

- uma tarefa por vez;
- mudanças limitadas ao escopo aprovado;
- testes focais e regressão;

- documentação aplicável atualizada;
- status e checkboxes sustentados por evidência.

Erros comuns

- implementar com gate pendente;
- aceitar um RED causado por dependência ausente;
- ampliar o escopo sem atualizar a spec;
- marcar conclusão antes da regressão;
- esquecer documentação ou contexto afetado.

Próximo passo

Continue com a próxima tarefa ou consulte [specsfy-base-progress](#) . Se surgir um pedido novo, use [specsfy-base-update-spec](#) .

Incorporar mudanças com `specsfy-base-update-spec`

Esta skill atualiza uma spec que já foi definida, planejada, iniciada ou concluída. O pedido novo entra na mesma fonte, e somente os atos afetados são reabertos.

Quando usar

Use quando você disser “esqueci”, “adicione”, “remova”, “corrija” ou “mude” algo de uma entrega existente.

Para criar a primeira versão da spec, use `specify`.

Como pedir

Use `$specsfy-base-update-spec` para adicionar expiração de 30 minutos à `specs/specs/0004-recuperar-senha/spec.md`.

Você também pode pedir uma remoção:

Remova a exigência de SMS da spec 0004 e ajuste o trabalho afetado.

Exemplo passo a passo

1. A skill preserva literalmente o pedido novo.
2. Lê requisitos, testes, tarefas, gates e evidências atuais.
3. Classifica a mudança:

Mudança de definição: altera comportamento e critério de aceite.
Atos reabertos: I, II e III.

1. Atualiza a mesma `spec.md`.
2. Invalida apenas os gates que perderam validade.
3. Retoma entrevista, validação, tarefas, testes e implementação na ordem necessária.
4. Registra a nova evidência sem apagar o histórico relevante.

O que esperar

- pedido original preservado;

- impacto explicado antes da retomada;
- nenhuma spec duplicada;
- trabalho já válido mantido;
- transições automáticas até o estado coerente.

Erros comuns

- editar apenas o código e deixar a spec antiga;
- reabrir todos os gates sem analisar impacto;
- manter teste ou tarefa que contradiz o pedido novo;
- esconder a mudança em notas soltas;
- criar uma segunda especificação para a mesma entrega.

Próximo passo

A própria skill encaminha para a etapa reaberta. Depois da atualização, use [specsfy-base-progress](#) para revisar o estado geral.

Consultar o estado com `specsify-base-progress`

Esta skill lê todas as specs e apresenta uma visão geral de gates, tarefas, checklists, bloqueios e próximo trabalho. Ela não altera nenhum estado.

Quando usar

Use para perguntar “quanto falta?”, “o que está bloqueado?”, “qual é a próxima tarefa?” ou “quais specs estão concluídas?”.

Como pedir

Na conversa:

```
Use $specsify-base-progress para mostrar o progresso do projeto.
```

Pelo CLI:

```
specsify progress --project .
```

Exemplo passo a passo

1. A skill lê `specs/specs/*/spec.md`.
2. Calcula o estado a partir de gates e checkboxes.
3. Não consulta um relatório paralelo.
4. Apresenta:

```
Specs: 2
Complete: 1
Implementing: 1
Tarefas: 7 de 10 concluídas
Próximo trabalho: T004 da spec 0004-recuperar-senha
Bloqueios: nenhum
```

Se os metadados estiverem incoerentes, o relatório mostra a pendência em vez de inventar um percentual confiável.

O que esperar

- leitura somente das fontes canônicas;

- totais de specs e tarefas;
- gates pendentes;
- bloqueios e próximo trabalho;
- saída humana ou JSON pelo CLI.

Erros comuns

- alterar checkboxes durante a consulta;
- manter um segundo arquivo de progresso;
- calcular conclusão só pelo número de arquivos;
- esconder spec inválida do relatório;
- confundir progresso com autorização para implementar.

Próximo passo

Abra a página da skill indicada pelo próximo trabalho. Para acompanhar continuamente no terminal, use:

```
specsfy progress --project . --watch
```

CLI e TUI do Specsify

Papel

Explicar como operar e atualizar o executável `specsify` depois do bootstrap, sem transformar o workspace de desenvolvimento em projeto consumidor.

Como usar

Instale primeiro o executável e o framework seguindo o [guia de instalação](#). Este guia assume que `specsify --version` já responde no terminal e que o bootstrap foi executado no projeto consumidor. Para conduzir a primeira fatia depois do bootstrap, siga o [guia do primeiro projeto](#). Para seleção técnica, automação e reabertura de gates, consulte o [uso avançado](#).

Os templates de ideia, backlog, spec, tarefas e contexto persistente ficam centralizados em `.specsify/templates/`. Ao criar uma especificação, `specsify-base-specify` renderiza o template instalado em `specs/specs/<NNNN>-<slug>/spec.md`. O arquivo de exemplo demonstra os três atos e as 18 seções para agentes, testes e diagnóstico do CLI, mas não é fonte normativa de uma feature. O cabeçalho renderizado é uma tabela Markdown de duas colunas, `Campo` e `Valor`, com um metadado por linha.

Instale as bases e todos os especialistas detectados em uma única chamada:

```
specsify install --project . --detected
```

Também é possível selecionar especialistas explicitamente. Repita `--specialist` para compor o conjunto:

```
specsify install --project . \  
  --specialist specsify-specialist-laravel \  
  --specialist specsify-specialist-postgres
```

Para consultar e gerenciar o catálogo separadamente:

```
specsify skills list  
specsify skills detect --project .  
specsify skills add specsify-specialist-laravel --project .  
specsify skills update --project .
```


Atualização automática

Ao abrir `specsify` ou `specsify tui` em um terminal interativo, o CLI verifica as tags semânticas estáveis de `cli/`. O cache e as configurações globais ficam em `~/.specsify/cli.json`, com permissão restrita ao usuário e intervalo padrão de 24 horas entre consultas.

Quando uma tag aponta para uma versão superior, o CLI apresenta a versão atual e pergunta se deve atualizar. Recusar abre o dashboard normalmente. Aceitar executa `uv tool upgrade specsify-cli`, preservando a origem e as opções registradas pelo `uv` na instalação, e então encerra. A versão nova entra em uso na próxima abertura.

O arquivo global separa `settings`, incluindo habilitação e intervalo da consulta, de `cache`, que registra horário, tag, versão, commit, ETag e erro recente. Chaves desconhecidas são preservadas. Indisponibilidade de rede, resposta inválida ou falha de escrita não bloqueiam a aplicação; o aviso é adiado e o fluxo normal continua.

Como o monorepo é privado, catálogo e tags são consultados com `GH_TOKEN`, `GITHUB_TOKEN` ou, na ausência dessas variáveis, com a sessão de `gh auth token`. Execute `gh auth login` antes do primeiro uso. O token não é copiado para `~/.specsify/cli.json`.

A mesma atualização pode ser iniciada diretamente, sem abrir a TUI:

```
uv tool upgrade specsify-cli
```

Dashboard e progresso

Execute sem argumentos dentro do projeto consumidor para abrir a TUI:

```
specsify
```

`specsify tui --project PATH` abre explicitamente outro projeto. O dashboard é organizado em seis abas:

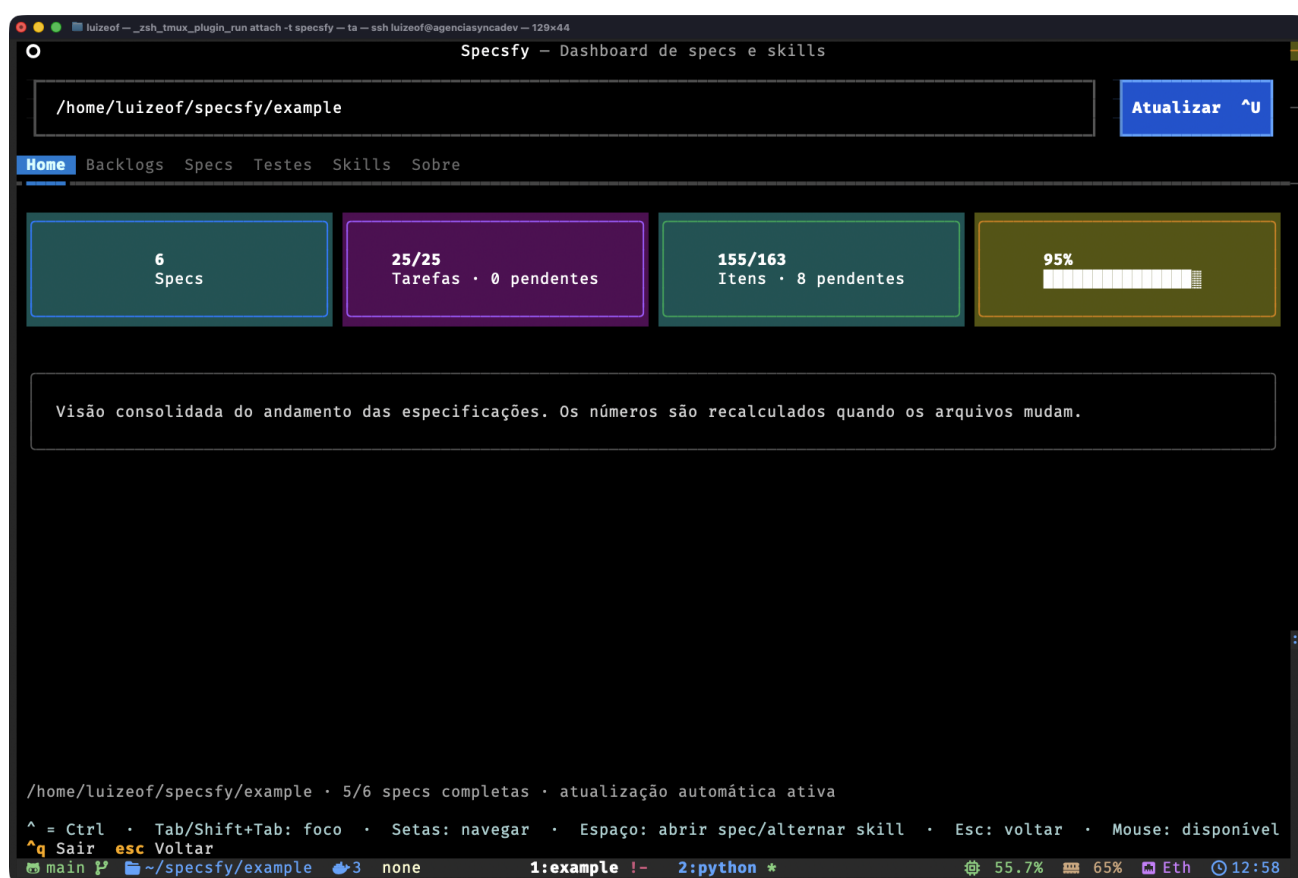
- **Home**, com as estatísticas consolidadas;
- **Backlogs**, com lista navegável na coluna esquerda e preview Markdown formatado na coluna direita;
- **Specs**, com a tabela e o progresso de cada especificação;
- **Testes**, com execução do runner detectado, resumo da última execução e saída detalhada em subabas separadas;
- **Skills**, com catálogo tabular, painel de detalhes e uma prévia explícita das instalações e remoções pendentes;
- **Sobre**, com a versão e a finalidade do CLI.

O dashboard apresenta:

- quantidade total e concluída de specs;
- tarefas T... concluídas, pendentes e totais;
- todos os itens de checklist concluídos, pendentes e totais;
- gates aprovados por spec;
- porcentagem e barra de progresso global;
- porcentagem e barra de progresso de cada spec.

Percurso visual

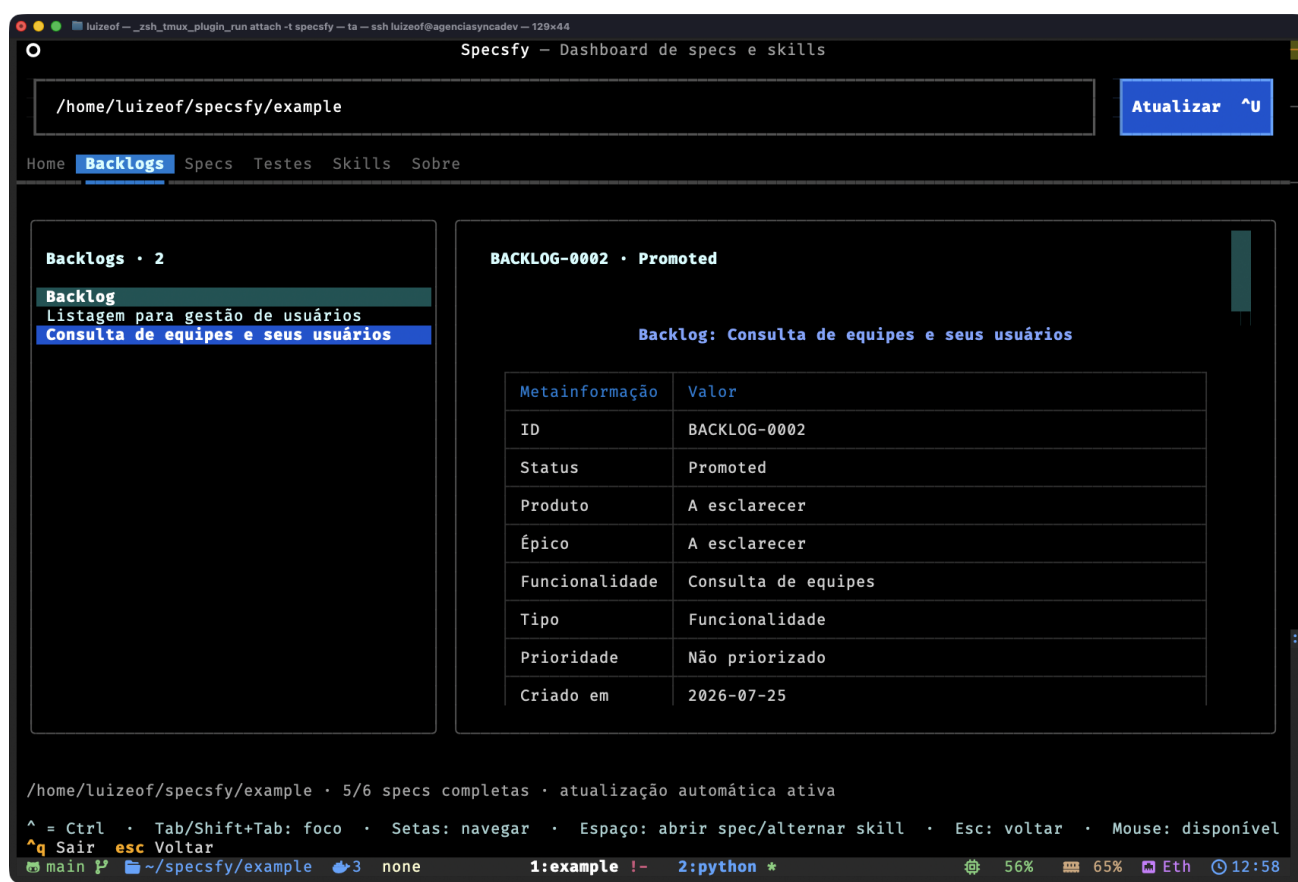
Home: visão consolidada



Dashboard Home

A Home reúne o total de specs, o avanço das tarefas e checklists e a porcentagem global. O diretório selecionado aparece no topo e o rodapé confirma quantas specs estão completas e se a atualização automática está ativa.

Backlogs: lista e leitura lado a lado



Specsfy — Dashboard de specs e skills

/home/luizeof/specsfy/example Atualizar ^U

Home **Backlogs** Specs Testes Skills Sobre

Backlogs · 2

Backlog

- Listagem para gestão de usuários
- Consulta de equipes e seus usuários**

BACKLOG-0002 · Promoted

Backlog: Consulta de equipes e seus usuários

Metainformação	Valor
ID	BACKLOG-0002
Status	Promoted
Produto	A esclarecer
Épico	A esclarecer
Funcionalidade	Consulta de equipes
Tipo	Funcionalidade
Prioridade	Não priorizado
Criado em	2026-07-25

/home/luizeof/specsfy/example · 5/6 specs completas · atualização automática ativa

^ = Ctrl · Tab/Shift+Tab: foco · Setas: navegar · Espaço: abrir spec/alternar skill · Esc: voltar · Mouse: disponível

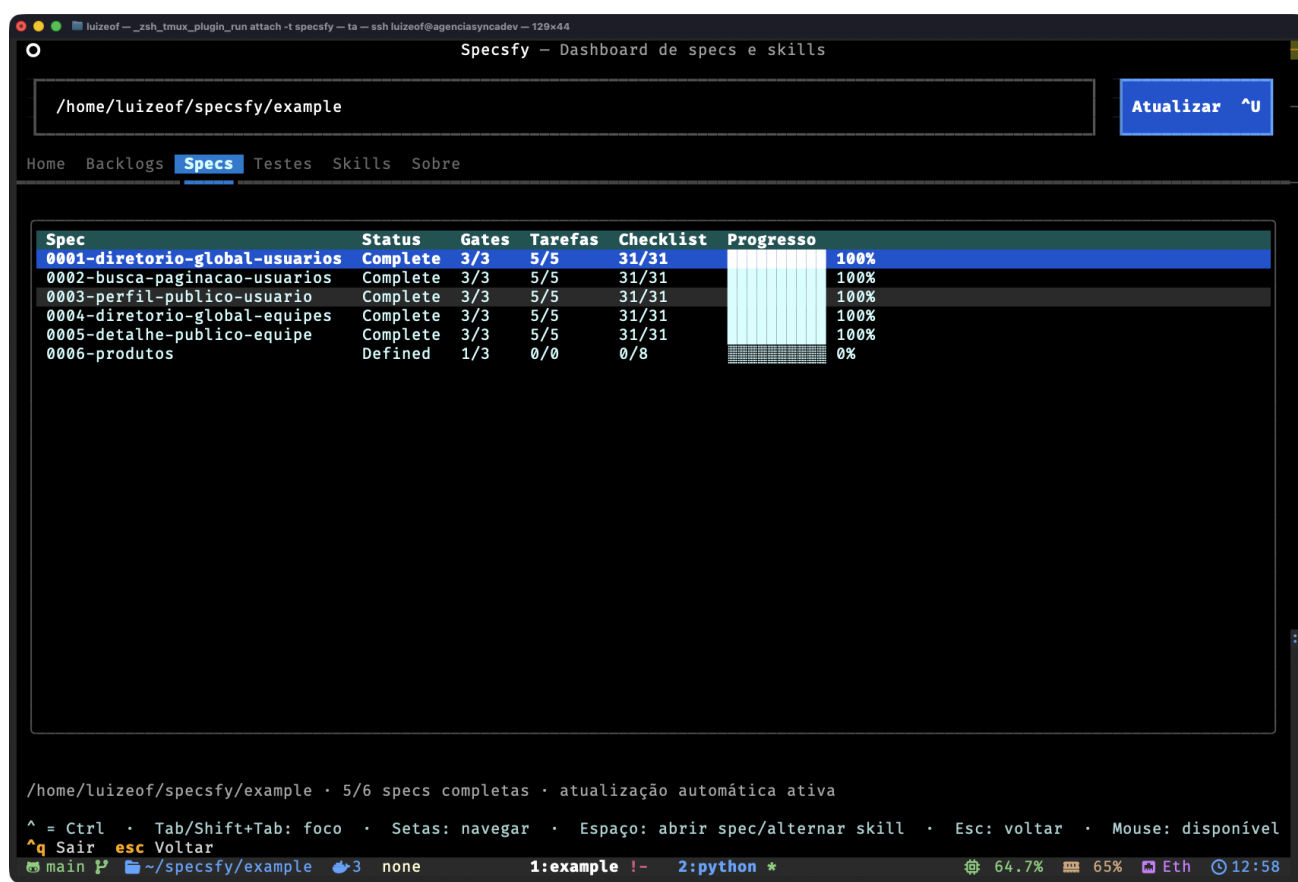
q Sair esc Voltar

main ~/specsfy/example 3 none 1:example !- 2:python * 56% 65% Eth 12:58

Backlogs

A aba Backlogs mantém a seleção na coluna esquerda e renderiza o Markdown do item na direita. Assim é possível conferir metainformação, status e conteúdo sem abandonar o dashboard.

Specs: gates, tarefas e progresso



The screenshot shows the Specsfy dashboard with the following table:

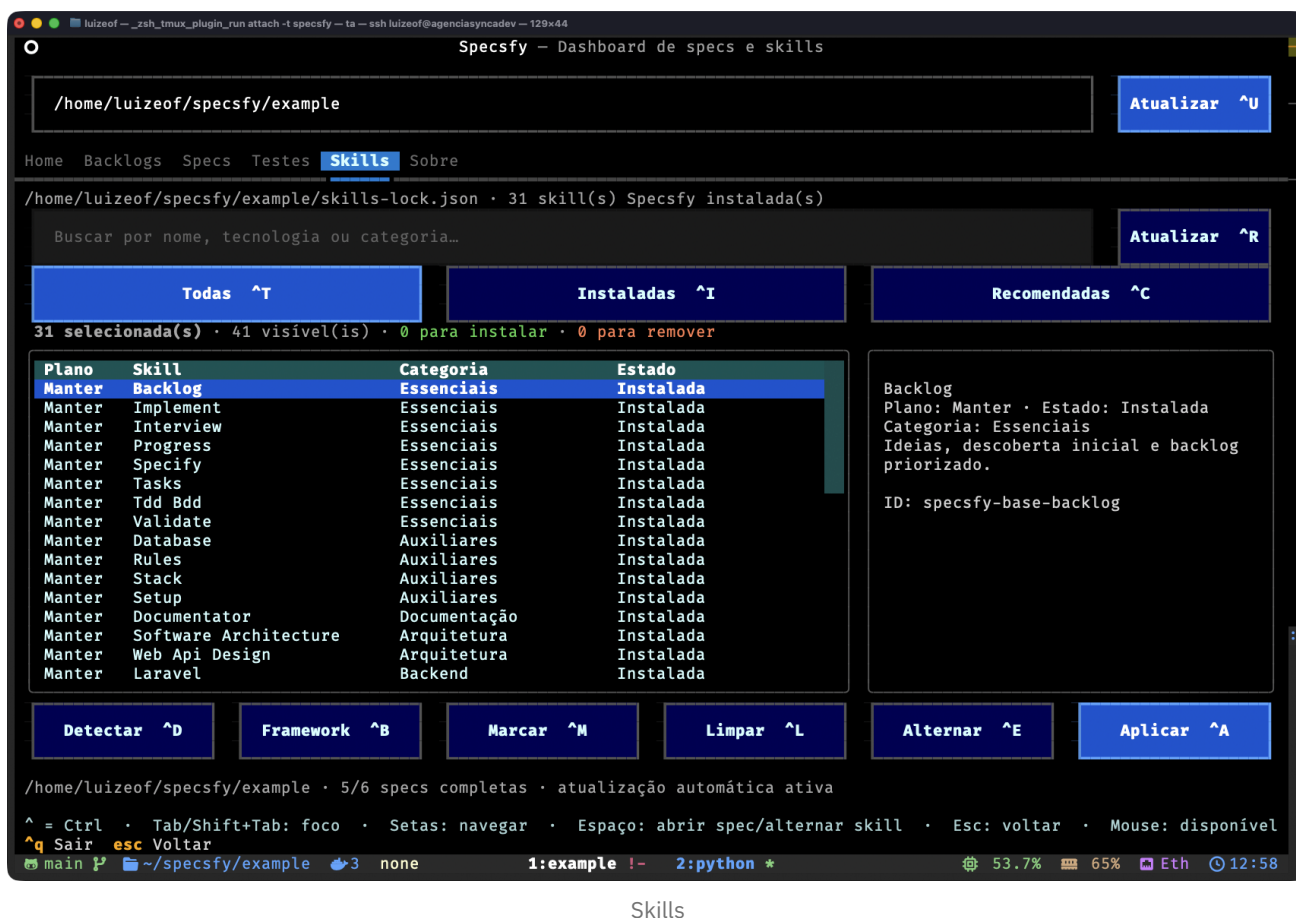
Spec	Status	Gates	Tarefas	Checklist	Progresso
0001-diretorio-global-usuarios	Complete	3/3	5/5	31/31	100%
0002-busca-paginacao-usuarios	Complete	3/3	5/5	31/31	100%
0003-perfil-publico-usuario	Complete	3/3	5/5	31/31	100%
0004-diretorio-global-equipes	Complete	3/3	5/5	31/31	100%
0005-detalle-publico-equipe	Complete	3/3	5/5	31/31	100%
0006-produtos	Defined	1/3	0/0	0/8	0%

At the bottom of the dashboard, there is a status bar showing: 5/6 specs completas · atualização automática ativa. Below this, a legend indicates: ^ = Ctrl, Tab/Shift+Tab: foco, Setas: navegar, Espaço: abrir spec/alternar skill, Esc: voltar, Mouse: disponível. The bottom status bar also shows: main, ~/specsfy/example, 3, none, 1:example, 2:python, 64.7%, 65%, Eth, 12:58.

Specs

A aba Specs compara status, gates, tarefas, checklists e porcentagem por especificação. A linha destacada pode ser aberta com `Espaço` para consultar a spec completa em um modal Markdown.

Skills: planejar antes de aplicar



Skills

A aba Skills combina busca, filtros, plano, categoria e estado com o painel de detalhes da seleção. Os totais acima da tabela antecipam instalações e remoções; nada muda no projeto até a ação **Aplicar**.

Testes do projeto

Em um projeto Laravel com Pest, execute:

```
specsify test --project .
```

O CLI detecta `artisan` e `pestphp/pest`, chama `php artisan test` diretamente na raiz selecionada, transmite a saída e preserva o exit code do runner. Ele não aceita uma string de shell arbitrária.

Na aba **Testes**, Executar testes `^X` inicia a mesma execução. A subaba **Resumo** mostra resultado, runner, comando, projeto, duração, exit code e os totais emitidos pelo Pest. A subaba **Testes** mantém a saída completa e rolável; quando o projeto fornece um relatório Pest estruturado, cada falha é apresentada com nome do teste, arquivo, linha e mensagem.

O progresso usa todos os checkboxes Markdown de `specs/specs/*/spec.md`. Tarefas com ID `T...` também ganham estatística própria. Quando uma spec não possui checkboxes, os três gates são usados como projeção de fallback.

A TUI calcula fingerprints dos backlogs, das specs e do `skills-lock.json`. Ela atualiza automaticamente as listas, previews, cards e seleção de skills quando esses arquivos mudam. O intervalo padrão é 0,75 segundo e pode ser configurado por projeto:

```
specsify config show --project .  
specsify config set --project . --watch-interval 0.5
```

O rodapé apresenta os atalhos:

- `Ctrl+Q` : sair;
- `Ctrl+U` : atualizar;
- `Ctrl+D` : detectar recomendações;
- `Ctrl+B` : selecionar todas as skills do framework;
- `Ctrl+E` : alternar o plano da skill destacada;
- `Ctrl+M` : marcar os resultados visíveis;
- `Ctrl+L` : limpar os resultados visíveis;
- `Ctrl+A` : aplicar a seleção;
- `Ctrl+R` : atualizar todas as skills Specsify instaladas;
- `Ctrl+T` , `Ctrl+I` , `Ctrl+C` : abrir os filtros Todas, Instaladas e Recomendadas;
- `Ctrl+H` , `Ctrl+G` , `Ctrl+S` , `Ctrl+K` , `Ctrl+O` : abrir Home, Backlogs, Specs, Skills e Sobre.
- `Ctrl+J` : abrir Testes;
- `Ctrl+X` : executar os testes do projeto selecionado.

Os atalhos são globais e aparecem nos próprios rótulos dos botões. A interface inteira também aceita:

- `Tab` e `Shift+Tab` para percorrer controles;
- setas para navegar listas e tabelas;
- `Enter` ou `Espaço` para alternar o plano da skill destacada;
- `Esc` para limpar a busca ou voltar à Home;
- mouse para abas, listas, preview, filtros e botões.

Foco, cursor, seleção e ações primárias usam contraste reforçado para manter legibilidade em terminais escuros.

Na aba Skills, cada linha separa `Plano` , `Skill` , `Categoria` e `Estado` . O plano usa os valores `Instalar` , `Manter` , `Remover` e `Ignorar` ; ele expressa o que acontecerá sem executar a mudança imediatamente. O painel lateral mostra o identificador completo, a descrição, a recomendação e o plano da linha destacada. A alteração só ocorre ao acionar `Aplicar` .

A configuração vive em `<projeto>/specsify/config.json` . Valores desconhecidos adicionados pelo usuário são preservados quando o CLI atualiza uma opção.

Para automação e integração com outras ferramentas:

```
specsify progress --project .
specsify progress --project . --json
specsify progress --project . --watch
specsify progress --project . --watch --interval 0.5 --json
```

O JSON contém um objeto `summary` e a coleção `specs`. Com `--watch`, um novo snapshot é emitido somente quando o conteúdo das specs muda. A TUI também oferece instalação das bases, detecção e instalação de especialistas.

O leitor mantém compatibilidade com `specs/<NNNN>-<slug>/spec.md` para projetos existentes, mas toda spec nova usa `specs/specs/`. Itens em `specs/backlog/` não entram na porcentagem de entrega.

Justificativa de tamanho

Este guia permanece em uma unidade porque comandos equivalentes, atualização, dashboard, atalhos e segurança formam a mesma interface pública do CLI. A instalação possui jornada e pré-requisitos próprios em `installation.md`; cada capacidade operacional permanece em seção própria para leitura localizada.

Atualize quando

- um comando, argumento, destino ou formato público mudar;
- a TUI ou o mecanismo de atualização mudar;
- o contrato de automação ou progresso mudar.

Não use para

- instalar no workspace `promovaweb/specsify`;
- criar specs ou alterar gates;
- documentar a implementação interna do Python;
- executar deploy ou instalar pacotes de aplicação.

Fonte da verdade e precedência

O comportamento executável vive em `cli/`. As skills base pertencem a `skills/`, os especialistas a `specialists/` e cada spec ao projeto consumidor.

Segurança e reversibilidade

- O destino padrão é `<projeto>/agents/skills`.

- As regras centrais ficam em `<projeto>/specsify/Spec.md`.
- Template e exemplo ficam em `<projeto>/specsify/templates/Spec.md` e `<projeto>/specsify/examples/Spec.md`.
- `AGENTS.md` e `CLAUDE.md` recebem somente blocos delimitados e atualizáveis.
- O registro fica em `<projeto>/specsify/skills-lock.json`.
- Cada skill gerenciada registra um fingerprint de conteúdo no lock.
- Uma execução repetida sobre a mesma versão não altera arquivos nem o lock.
- Uma skill gerenciada e intacta pode receber uma versão nova sem `--force`.
- Alterações locais bloqueiam atualização e remoção; `--force` é a decisão explícita para descartá-las.
- Downloads dos catálogos usam checkout temporário; a materialização é delegada a `skills add`.
- `specsify skills remove <nome>` remove somente o nome explícito e preserva conteúdo local divergente.
- O CLI recusa a raiz reconhecida do workspace `promovaweb/specsify`.

Contexto persistente do projeto

Papel

Explicar como manter uma descrição durável do projeto, seu stack, suas regras e sua persistência para que pessoas e agentes compartilhem o mesmo contexto sem criar uma segunda spec.

Como usar

Execute `$specsify-setup` depois de instalar o framework e sempre que quiser verificar sua consistência. A skill detecta Laravel, Next.js e Astro por seus manifests, sugere um modelo apropriado e garante esta estrutura:

```
<projeto>/
├── PROJECT.md
├── .specsify/
│   ├── STACK.md
│   ├── RULES.md
│   └── DATABASE.md
```

Ela também reserva blocos delimitados para as diretrizes do Specsify em `AGENTS.md` e `CLAUDE.md`. Conteúdo fora desses blocos pertence ao usuário e é preservado. A referência publicável das diretrizes vive em `specsify-setup`.

ARQUIVO	CONTEÚDO	SKILL MANTENEDORA
PROJECT.md	história, finalidade, pessoas, capacidades e limites	<code>\$specsify-setup</code> cria o modelo; a equipe mantém a narrativa
.specsify/STACK.md	tecnologias estruturais e evidências	<code>\$specsify-aux-stack</code>
.specsify/RULES.md	regras explícitas confirmadas	<code>\$specsify-aux-rules</code>
.specsify/DATABASE.md	fontes, tabelas, campos, relações e migrations	<code>\$specsify-aux-database</code>

Os quatro modelos ficam em `.specsify/templates/Project.md`, `Stack.md`, `Rules.md` e `Database.md`, junto dos demais templates do framework.

Execute `$specsify-aux-stack` após alterar frameworks, runtimes, ferramentas estruturais ou persistência. Execute `$specsify-aux-database` sempre que criar ou alterar banco, schema, tabela, coleção, model persistente, campo, relação, índice ou migration. Use `$specsify-aux-rules` para formular e acrescentar uma regra confirmada sem duplicar ou apagar regras anteriores.

Monitoramento durante mudanças

O monitor é uma verificação executada pelas skills durante o fluxo, não um daemon em segundo plano. Ele examina caminhos staged, unstaged e untracked:

```
python3 -B .agents/skills/specsfy-setup/scripts/monitor_context.py \
  --project . --check
```

SINAL OBSERVADO	OBRIGAÇÃO
manifest, lockfile ou configuração estrutural	atualizar <code>.specsfy/STACK.md</code> com <code>\$specsfy-aux-stack</code>
banco, schema, model persistente, tabela, campo ou migration	atualizar <code>.specsfy/DATABASE.md</code> com <code>\$specsfy-aux-database</code>
código da aplicação	revisar <code>PROJECT.md</code>
código da aplicação ou persistência	reconstruir <code>docs/</code> com <code>\$specsfy-documentator</code>
instrução ou convenção	revisar <code>.specsfy/RULES.md</code> com <code>\$specsfy-aux-rules</code>

`PENDING` impede a conclusão da tarefa e do Delivery Gate. Quando uma mudança de aplicação não altera história, finalidade, capacidades ou limites, registre essa avaliação na evidência da tarefa e execute novamente com `--acknowledge-project-no-change`. Para uma revisão de regras sem regra nova, use `--acknowledge-rules-no-change` somente depois de registrar a justificativa. Veja a topologia e o `--check` no guia de [documentação técnica do sistema](#).

Atualize quando

- a localização ou finalidade de um dos quatro arquivos mudar;
- uma skill mantenedora ganhar ou perder responsabilidade;
- a política de preservação dos blocos gerenciados mudar;
- um novo stack receber modelo próprio no setup.
- os sinais monitorados ou a política de bloqueio documental mudarem.

Não use para

- substituir `specs/specs/<NNNN>-<slug>/spec.md`;
- registrar tarefas, gates ou aceite de uma fatia;
- copiar segredos, dados de produção ou valores de `.env`;
- tratar inferência de uma ferramenta como regra confirmada pelo usuário.

Fonte da verdade e precedência

Schemas, migrations, manifests, lockfiles e configurações comprovam o estado implementado. Os quatro arquivos sintetizam esse estado e o conhecimento humano durável. A spec continua governando comportamento e mudança; `AGENTS.md` e `CLAUDE.md` governam instruções dos agentes. Em divergência, preserve o conteúdo observado, sinalize o conflito e corrija a fonte proprietária antes da síntese.

Preservação e atualização

- O setup cria um arquivo de contexto somente quando ele ainda não existe.
- As auxiliares atualizam apenas blocos detectados delimitados e preservam seções humanas.
- Uma nova varredura nunca autoriza remover silenciosamente uma decisão humana.
- `DATABASE.md` usa tabelas Markdown para facilitar mapeamento e comparação.
- Valores sensíveis não são lidos nem registrados; cite somente nomes seguros de variáveis e caminhos das fontes.

Documentação técnica do sistema

Papel

Explicar como reconstruir uma visão técnica completa de uma aplicação existente em `<projeto>/docs/`. Essa documentação pertence ao projeto consumidor e não se confunde com este repositório, que documenta a metodologia Specsify.

Como usar

Execute `$specsify-documentator` livremente para documentar um sistema legado ou atualizar sua visão técnica. O handoff também é obrigatório depois de cada tarefa de código concluída por `$specsify-base-implement`.

A skill lê o código completo, manifests, locks, rotas, migrations, testes, configuração e o contexto persistente. Cada execução reconstrói blocos delimitados nos seguintes arquivos, preservando texto humano externo:

ARQUIVO NO CONSUMIDOR	CONTEÚDO
<code>docs/README.md</code>	portal e ordem de leitura
<code>docs/architecture.md</code>	componentes, dependências e UML Mermaid
<code>docs/application.md</code>	módulos e implementações observadas
<code>docs/database.md</code>	entidades, campos, relações e <code>erDiagram</code>
<code>docs/flows.md</code>	rotas, <code>flowchart</code> e <code>sequenceDiagram</code>
<code>docs/testing.md</code>	runners, comandos, inventário e resumo
<code>docs/frontend.md</code>	views, páginas, componentes, React e Tailwind
<code>docs/packages.md</code>	runtime, framework, nativos, integrados e terceiros
<code>docs/integrations.md</code>	serviços externos e nomes de configuração
<code>docs/decisions.md</code>	decisões explícitas e suas fontes

Laravel inclui rotas, controllers, models, services, jobs, policies, Blade, migrations e Pest/PHPUnit. Node, Next.js, React e Astro incluem páginas, APIs, componentes, módulos, scripts e Vitest, Jest ou Node Test quando observados. Pacotes registram versão e link do repositório GitHub; quando a fonte não puder ser confirmada localmente, a saída usa uma busca rotulada e não inventa URL.

Depois da reconstrução, a própria skill executa:

```
python3 -B .agents/skills/specsfy-documentator/scripts/build_documentation.py \
  --project . --check
```

O monitor do setup também bloqueia a entrega quando código de aplicação ou persistência mudou e nenhum arquivo de `docs/` foi reconstruído.

Atualize quando

- a topologia documental gerada mudar;
- um framework, runner, diagrama ou classe de inventário passar a ser coberto;
- o handoff entre implementação, monitor e documentador mudar.

Não use para

- substituir `spec.md`, `PROJECT.md` ou arquivos `.specsfy/`;
- copiar segredos, valores de ambiente, dados de produção ou código integral;
- declarar uma inferência como decisão confirmada;
- manter documentação oficial da metodologia dentro do projeto consumidor.

Fonte da verdade e precedência

O código, os testes, os manifests, os schemas e as migrations comprovam o estado implementado. A spec governa o comportamento da fatia; `PROJECT.md` e `.specsfy/` preservam o contexto transversal do consumidor. Os arquivos em `<projeto>/docs/` são uma projeção reconstruível dessas fontes.

Atualizar uma especificação

Papel

Permitir que uma pessoa adicione, remova, corrija ou mude um pedido sem conhecer os atos internos do Specsify e sem deixar código, testes e tarefas divergirem da especificação.

Como usar

Quando lembrar de algo durante ou depois da implementação, diga diretamente o que mudou. Use esta entrada para adicionar, remover, corrigir ou mudar um pedido:

```
Use $specsify-base-update-spec em
specs/specs/0001-pagina-boas-vindas/spec.md:
esqueci de pedir que o nome tenha no máximo 80 caracteres.
```

Também funcionam pedidos naturais como:

```
Quero adicionar uma regra à especificação atual.
Remova o comportamento de visitante.
Corrija o que acontece quando o nome estiver vazio.
Mude esta entrega para exigir autenticação.
```

A skill preserva o pedido, compara-o com a spec existente e informa primeiro o resultado prático. Você não precisa escolher manualmente qual gate ou etapa reabrir.

O que acontece

TIPO DO PEDIDO	TRATAMENTO
correção interna sem efeito observável	mantém a spec e registra a avaliação na evidência
esclarecimento sem mudança de significado	ajusta a redação sem invalidar gates
comportamento, aceite, escopo, dados ou segurança	atualiza a mesma spec e reabre desde o Ato I
solução, tarefas ou estratégia de testes	atualiza a mesma spec e reabre desde o Ato II
capacidade independente	preserva a spec atual e encaminha o pedido para backlog ou nova spec
decisão material ausente	faz uma pergunta objetiva e retoma a atualização depois da resposta

Quando a mudança pertence à spec atual, o fluxo é automático:

```
pedido tardio → update-spec → validate ou tasks → TDD/BDD → implement → progress
└─ interview, somente se faltar uma decisão
```

Testes e tarefas que continuam válidos são preservados. Provas dependentes do comportamento anterior voltam a ficar pendentes; a implementação só é retomada depois de existir novo RED válido e os gates necessários voltarem a `Passed`.

Resultado esperado

- o pedido novo está incorporado à única `spec.md` normativa;
- IDs e decisões ainda válidos foram preservados;
- os gates afetados foram reabertos;
- validação, tarefas e testes foram reconciliados;
- a etapa que recebeu o pedido foi retomada automaticamente.

Limites

- não cria `change-request.md`, `plan.md`, `tasks.md` ou outra fonte paralela;
- não transforma uma capacidade independente em aumento silencioso de escopo;
- não inventa uma decisão material;
- não altera código antes de atualizar a spec e preparar novo RED;
- handoffs automáticos não autorizam deploy, publicação ou ação destrutiva.

Atualize quando

- a classificação de mudanças tardias mudar;
- a política de invalidação dos gates mudar;
- a responsabilidade ou o nome de `specsfy-base-update-spec` mudar.

Não use para

- criar a primeira versão de uma spec;
- capturar uma ideia ainda superficial;
- editar código sem mudança normativa;
- manter histórico fora da spec.

Fonte da verdade e precedência

A execução pertence a `specsfy-base-update-spec`. A spec atualizada permanece como única fonte normativa no projeto consumidor.

Skills especialistas

Papel

Separar o método `specsfy-base-*` do contexto técnico `specsfy-specialist-*`, permitindo instalar somente o conhecimento necessário em cada projeto consumidor.

Como usar

Detecte tecnologias e revise a sugestão:

```
specsfy skills detect
```

Instale nomes explícitos:

```
specsfy skills add \  
  specsfy-specialist-laravel \  
  specsfy-specialist-postgres \  
  specsfy-specialist-redis
```

As skills base podem propor um especialista. Se ele já estiver instalado, anunciam a transição automática e o carregam na mesma conversa. Se estiver ausente, a instalação recebe autorização específica; o handoff não instala nada automaticamente.

Atualize quando

- uma skill entrar, sair ou mudar de responsabilidade;
- o prefixo, detecção ou modo de instalação mudar;
- padrões oficiais relevantes mudarem.

Não use para

- redefinir o fluxo dos três atos;
- criar uma spec ou substituir a fonte normativa do projeto;
- instalar todo o catálogo por padrão;
- copiar versões de dependências mantidas em manifests.

Fonte da verdade e precedência

Diretórios, `SKILL.md`, referências, metadata e `catalog.json` vivem em `specialists/`. Esta página orienta usuários sem duplicar o conteúdo operacional de cada skill.

Catálogo por domínio

DOMÍNIO	ESPECIALISTAS
backend e dados	Laravel, Supabase, Postgres, Redis e APIs web
frontend	React, Astro, Next.js, TypeScript e Tailwind CSS
interface	shadcn/ui, UI, UX e acessibilidade web
plataforma	Docker, Docker Swarm, Ansible e engenharia de entrega
qualidade	segurança, observabilidade e performance
design técnico	arquitetura e modelagem de domínio
engenharia	code review, debugging, prototipação, pesquisa e conflitos Git

Cada especialista inclui workflow, padrões, validação e referências oficiais. Use especialistas relacionados em conjunto apenas quando seus boundaries forem realmente tocados.

Relação com as bases

- `specsify-base-interview` identifica necessidade.
- `specsify-base-specify` registra requisitos e NFRs.
- `specsify-base-validate` seleciona lentes de revisão.
- `specsify-base-tasks` usa checklists técnicos para decompor trabalho.
- `specsify-base-tdd-bdd` preserva RED/GREEN.
- `specsify-base-implement` executa no contexto da stack.
- `specsify-base-update-spec` revisa requisitos e NFRs afetados por pedido tardio.
- `specsify-base-progress` identifica contexto e executa a transição automática.

Especialista complementa conhecimento; nunca avança gate por presença.

Uso avançado do Specsfy

Papel

Combinar o método base com especialistas, saída estruturada, monitoramento e reabertura disciplinada dos gates em projetos já operacionais.

Como usar

Pré-condições

- jornada do [primeiro projeto](#) compreendida;
- CLI e framework instalados;
- projeto consumidor selecionado explicitamente.

Detecte e instale contexto técnico

Veja as recomendações sem alterar o projeto:

```
specsfy skills detect --project .
```

Instale o framework e todos os especialistas detectados:

```
specsfy install --project . --detected
```

Ou componha uma seleção explícita:

```
specsfy install --project . \  
  --specialist specsfy-specialist-laravel \  
  --specialist specsfy-specialist-postgres
```

Em projeto já preparado, adicione somente especialistas:

```
specsfy skills add specsfy-specialist-laravel --project .
```

Deteção é recomendação baseada em manifests, dependências e marcadores do projeto. Revise a seleção antes de instalar. Especialistas orientam decisões técnicas; não criam specs nem avançam gates.

Automatize a leitura de progresso

```
specsify progress --project . --json  
specsify progress --project . --watch --interval 0.5 --json
```

O JSON contém `summary` e `specs`. Com `--watch`, um snapshot novo aparece somente quando o conteúdo das specs muda. Use essa saída em painéis e automação de leitura; ela não deve ser usada para editar gates.

Ajuste a atualização visual

```
specsify config show --project .  
specsify config set --project . --watch-interval 0.5
```

A configuração vive em `<projeto>/specsify/config.json` e preserva chaves desconhecidas. Consulte todos os recursos no [guia do CLI](#).

Atualize sem perder customizações

```
uv tool upgrade specsify-cli  
specsify skills update --project .
```

O CLI usa fingerprints para distinguir conteúdo gerenciado intacto de customização local. Divergência bloqueia atualização ou remoção. Use `--force` somente após revisar a diferença e decidir descartar o conteúdo protegido.

Reabra a entrada correta

Quando lembrar de algo depois da definição, use a entrada explícita:

```
Use $specsify-base-update-spec em  
specs/specs/<NNNN>-<slug>/spec.md:  
quero adicionar, remover, corrigir ou mudar este pedido.
```

Quando um requisito observável muda, a skill reabre a definição; Definition, Plan e Delivery Gate precisam de evidência nova. Quando apenas o plano técnico muda, ela reabre o Ato II e o Ato III. Uma entrada alterada invalida provas posteriores construídas sobre a versão anterior, mas preserva tarefas e evidências que continuam compatíveis.

As skills fazem transições e retomadas na mesma conversa. Esse handoff não amplia autorização para instalar, publicar, fazer deploy ou executar ação destrutiva.

Consulte [Atualizar uma especificação](#) para a classificação completa e exemplos em linguagem comum.

Resultado esperado

O projeto mantém framework e especialistas deliberadamente selecionados, automação somente de leitura e gates coerentes com a versão atual da entrada.

Limites

- `--force` pode descartar customizações protegidas;
- `--detected` depende do catálogo e do estado observado no projeto;
- a TUI e o progresso projetam estado, não substituem a spec;
- especialista não substitui descoberta de versão, convenções ou riscos locais.

Atualize quando

- argumentos públicos de seleção, progresso ou configuração mudarem;
- proteções de atualização ou remoção mudarem;
- a política de reabertura dos atos mudar.

Não use para

- substituir o tutorial de primeira instalação;
- escolher especialista sem observar o projeto;
- automatizar escrita em gates ou specs.

Fonte da verdade e precedência

Comandos e proteções pertencem a `cli/`, o método base a `skills/` e o catálogo opcional a `specialists/`.

Usar Specsify com Laravel

Papel

Adicionar critérios Laravel ao fluxo Specsify sem substituir a spec da fatia, as convenções do projeto ou a versão comprovada pelos manifests.

Como usar

Pré-condições e detecção

O catálogo detecta Laravel por `artisan`, `composer.json` ou pela dependência `laravel/framework`. Confirme a versão e as extensões em `composer.json` e `composer.lock` antes de usar uma API do framework.

Instalação

Na raiz do projeto:

```
specsify install --project . --detected
```

Confira a recomendação antes, se preferir:

```
specsify skills detect --project .
```

Para instalar explicitamente:

```
specsify skills add specsify-specialist-laravel --project .
```

Passo a passo de uso

1. Capture ou promova a ideia pelo [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-laravel` na fatia ativa.
3. Confirme a versão, extensões, convenções locais e o caminho da requisição.
4. Na definição e no plano, trate autorização, validação, transação, idempotência, filas, falhas e migrations quando aplicáveis.
5. Derive testes para caminho feliz, autorização, validação, efeitos e falhas.
6. Implemente controllers finos e mantenha regras no boundary adotado pelo projeto; inspecione consultas e N+1 quando cardinalidade importar.
7. Execute os checks existentes. Em Laravel com Pest, o CLI oferece:

```
specsify test --project .
```

O CLI detecta `artisan` e `pestphp/pest`, chama `php artisan test` e preserva o exit code. Ele não recebe uma string arbitrária de shell.

O que o especialista acrescenta

- contratos HTTP, Form Requests, policies, resources e bindings;
- Eloquent, eager loading, casts e transações conscientes;
- jobs idempotentes, tentativas, backoff e tratamento de falha;
- migrations compatíveis com volume, locks, rollback e deploy misto;
- verificação de queues, scheduler, cache, configuração e ambiente.

Resultado esperado

A spec continua sendo a fonte normativa, enquanto testes e implementação consideram os riscos Laravel observados no projeto e na versão instalada.

Limites

- não aplique a skill a PHP sem Laravel;
- não presumas APIs pela versão mais recente da documentação;
- não execute migration, deploy ou comando operacional sem autorização;
- não confie apenas na validação ou autorização da interface.

Atualize quando

- a detecção do catálogo ou o nome da skill mudar;
- o fluxo, os padrões ou a validação do especialista mudar;
- o runner Laravel exposto pelo CLI mudar.

Não use para

- documentar PHP sem Laravel;
- fixar uma versão de Laravel não comprovada pelo projeto;
- substituir a documentação operacional da aplicação.

Fonte da verdade e precedência

A skill e seus padrões pertencem a `specsfy-specialist-laravel`. O estado do projeto é comprovado por código, `composer.json`, `composer.lock`, testes e configuração locais.

Usar Specsify com Astro

Papel

Adicionar decisões de renderização, conteúdo, ilhas e performance ao fluxo Specsify de um site Astro.

Como usar

Pré-condições e detecção

O catálogo detecta Astro pela dependência `astro` em `package.json` ou pelos arquivos `astro.config.mjs` e `astro.config.ts`. Descubra no projeto a versão, o output mode, o adapter, as integrações e as fontes de conteúdo.

Instalação

```
specsify skills detect --project .  
specsify skills add specsify-specialist-astro --project .
```

Para instalar o framework e as recomendações detectadas em uma chamada:

```
specsify install --project . --detected
```

Passo a passo de uso

1. Conduza a ideia até a spec conforme o [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-astro` na fatia ativa.
3. Classifique cada rota afetada como estática, sob demanda ou endpoint.
4. Mantenha HTML estático por padrão e escolha uma diretiva `client:*` somente para a interação que precisa de hidratação.
5. Modele conteúdo com schemas e relações explícitas; defina cache, headers, imagens e metadados quando aplicáveis.
6. Crie testes derivados do Gherkin para rotas, conteúdo inválido e comportamento hidratado.
7. Execute `astro check`, a suíte do projeto, o build de produção e o preview no runtime do adapter disponível no projeto.

O que o especialista acrescenta

- escolha explícita entre static, on-demand, island e endpoint;

- proteção contra transporte de dados sensíveis para ilhas;
- validação de slugs, relações e conteúdo;
- semântica, canonical, sitemap e dados estruturados;
- inspeção de payload cliente, Core Web Vitals e compatibilidade do adapter.

Resultado esperado

A fatia preserva a fonte normativa Specsify e torna observáveis a estratégia de renderização, a hidratação necessária, os contratos de conteúdo e a validação no runtime alvo.

Limites

- não escolha SSR sem necessidade de sessão, personalização ou frescor;
- não suponha APIs de Node em adapters edge;
- não valide apenas no servidor de desenvolvimento;
- confirme scripts e comandos disponíveis no `package.json`.

Atualize quando

- a detecção do catálogo ou o nome da skill mudar;
- o fluxo, os padrões ou a validação do especialista mudar;
- o contrato público de instalação de especialistas mudar.

Não use para

- prescrever um adapter ou modo de saída sem evidência;
- duplicar a documentação oficial do Astro;
- substituir scripts e convenções do projeto consumidor.

Fonte da verdade e precedência

A skill pertence a `specsify-specialist-astro`. Versão, adapter, scripts e integrações pertencem aos manifests, lockfiles e configuração do projeto consumidor.

Usar Specsify com Next.js

Papel

Adicionar fronteiras server/client, cache, mutations, rotas e deploy ao fluxo Specsify de uma aplicação Next.js.

Como usar

Pré-condições e detecção

O catálogo detecta Next.js pela dependência `next` em `package.json` ou por `next.config.js`, `next.config.mjs` e `next.config.ts`. Confirme versão, App/Pages Router, runtime, destino de deploy e flags antes de planejar.

Instalação

```
specsify skills detect --project .  
specsify skills add specsify-specialist-nextjs --project .
```

Ou instale bases e recomendações detectadas:

```
specsify install --project . --detected
```

Passo a passo de uso

1. Conduza a ideia até a spec pelo [primeiro projeto](#).
2. Peça ao agente para usar `$specsify-specialist-nextjs` na fatia ativa.
3. Mapeie rota, layout, `loading`, `error`, `not-found` e boundaries de dados.
4. No App Router, mantenha componentes no servidor por padrão e mova ao cliente apenas o boundary que requer estado, eventos ou APIs do navegador.
5. Defina cache, revalidation, tags e comportamento dinâmico explicitamente.
6. Trate Server Actions e Route Handlers como superfícies públicas: valide autenticação, autorização e entrada em cada mutation.
7. Derive testes para estados, redirects, autorização, invalidação e isolamento de dados entre usuários.
8. Execute lint, typecheck, testes, build e runtime de produção conforme os scripts do `package.json`.

O que o especialista acrescenta

- controle da fronteira entre Server e Client Components;
- prevenção de segredos e módulos server-only no bundle cliente;
- análise de waterfalls, streaming e recuperação de erro;
- cache como contrato compatível com a versão instalada;
- metadata, assets, bundle, imagens, fontes e Web Vitals.

Resultado esperado

As decisões de renderização, cache e segurança ficam rastreadas pela spec e provadas por testes e build na versão e no router realmente usados.

Limites

- não presuma App Router ou semântica de cache sem confirmar a versão;
- não mova uma árvore inteira ao cliente por conveniência;
- não use middleware para lógica longa ou incompatível com o runtime;
- não assuma comportamento específico do host sem documentá-lo.

Atualize quando

- a detecção do catálogo ou o nome da skill mudar;
- o fluxo, os padrões ou a validação do especialista mudar;
- o contrato público de instalação de especialistas mudar.

Não use para

- escolher App Router, Pages Router ou runtime sem evidência;
- duplicar documentação versionada do Next.js;
- substituir scripts, configuração ou convenções da aplicação.

Fonte da verdade e precedência

A skill pertence a `specsify-specialist-nextjs`. Router, versão, runtime e scripts são comprovados por código, `package.json`, lockfile e configuração do projeto consumidor.

Créditos do Specsfy

Papel

Reconhecer projeto, manutenção e comunidade, encaminhando identidade, contribuição e autoria detalhada às fontes que podem permanecer atuais.

Como usar

Use este arquivo para atribuição resumida. Consulte o histórico Git para autoria detalhada, o owner de marca para ativos e o repositório contribuído para licenciamento e instruções locais.

Projeto e manutenção

Specsfy é um projeto da Promovaweb, criado e mantido por **Luiz Eduardo Oliveira Fonseca** com a comunidade.

Contato: contato@promovaweb.com.

Comunidade

Contribuições de documentação, código, testes, pesquisa e feedback fazem parte da evolução do projeto. O histórico do monorepo preserva autoria e participação com precisão; este guia não mantém uma lista manual que possa ficar desatualizada.

Identidade

Logos, cores, tipografia, voz, acessibilidade e regras de aplicação pertencem ao diretório `brand/`. Use os ativos e orientações desse owner ao apresentar o projeto.

Componentes relacionados

O instalador do Specsfy delega a instalação de skills ao projeto `vercel-labs/skills`. O CLI pode usar o executável `skills` ou o fallback por `npm`, conforme documentado no [guia de instalação](#).

Como contribuir

Escolha primeiro o owner no mapa dos módulos, leia as instruções locais e preserve as fronteiras Git. A documentação oficial fica neste repositório; comportamento executável e testes ficam em seus respectivos owners.

Atualize quando

- a atribuição pública ou o contato oficial mudar;
- a responsabilidade pela identidade mudar;
- uma dependência institucional precisar de reconhecimento.

Não use para

- manter uma lista manual de contribuidores;
- inferir licença ausente em um repositório;
- copiar histórico Git ou inventários de dependências.

Fonte da verdade e precedência

A entrada pública confirma os créditos do projeto. Autoria detalhada e licenciamento são comprovados pelo histórico e pelos arquivos de cada repositório; a identidade oficial pertence a `brand/`.